Resuelve los siguientes ejercicios y entrega el código fuente java tanto en un fichero .pdf como en ficheros de texto, en el .pdf capturas de pantalla (si se piden) y las respuestas a las preguntas que se plantean (si las hay).

EJERCICIOS PADAWAN (OBLIGATORIOS)

EJERCICIO 1. DEFINIR Y USAR UNA CLASE EN JAVA.

Necesitamos un programa que trabaje con fracciones. Recuerda que una fracción está formada por dos números enteros llamados numerador y denominador donde el denominador no puede ser 0. Crea una clase llamada **Fraccion** que encapsule el estado (enteros numerador y denominador) y el siguiente comportamiento:

- **Constructores sobrecargados**:
  - Construntor por defecto: crea la fracción 1/1
  - Constructor que acepta 2 enteros (numerador y denominador), verifica y simplifica.
  - Constructor que acepta un double: Por ejemplo: 3.16 y genera la siguiente fracción:
    - 316/100 -> que tras simplificar queda 79/25
- **Getters y setters**: vigila (aserciones/excepciones) si denominador queda a 0. No permitas que en el denominador haya un número negativo (lo quitas cambiando el signo a numerador y denominador).
- **Operación interna (no se puede utilizar desde el exterior) simplifica()**: busca el máximo común divisor del valor absoluto de numerador y denominador y divide a ambos por él. Si el denominador es negativo, cambia el signo tanto a numerador como a denominador.
- **Operaciones externas (a nivel de objeto)**: suma(Fraccion f) y suma(int f) devuelven una nueva fracción (y no modifican la acctual) resultado de sumar a la actual, la que reciben como argumento. En el caso de recibir el entero f, lo interpretan como f/1. Recuerda que:

$$\frac{a}{b} + \frac{c}{d} = \frac{a * d + b * c}{b * d}$$ --> y luego simplificar.

- **Sobrecarga métodos de Object**: implementa versiones sobrecargadas de **equals()** y **toString()**. Ayúdate de la anotación @Override para asegurarte de que haces la sobrecarga de manera correcta. Para convertir a String contempla estos casos:

$\frac{a}{1}$ --> "a" $\frac{0}{b}$ --> "0" $\frac{a}{b}$ --> " a / b "

Prueba la clase haciéndola ejecutable, usa los 3 constructores, crea una cuarta fracción sumando dos de ellas e imprime las cuatro y compara dos que sean iguales y dos que sean diferentes...

Ejemplo:

```
C:\java Fraccion.java
Primera: 1
Segunda: 1/4
Equivalente de 0.25: 1/4
Numerador de 1/4 es 1
Denominador de 1/4 es 4
La suma de 1/4 y 1/4 es 1/2
1/2 es igual a 1/2: true
1/4 es igual a 1/2: false
```

EJERCICIO 2. UNA CLASE SIN ESTADO.

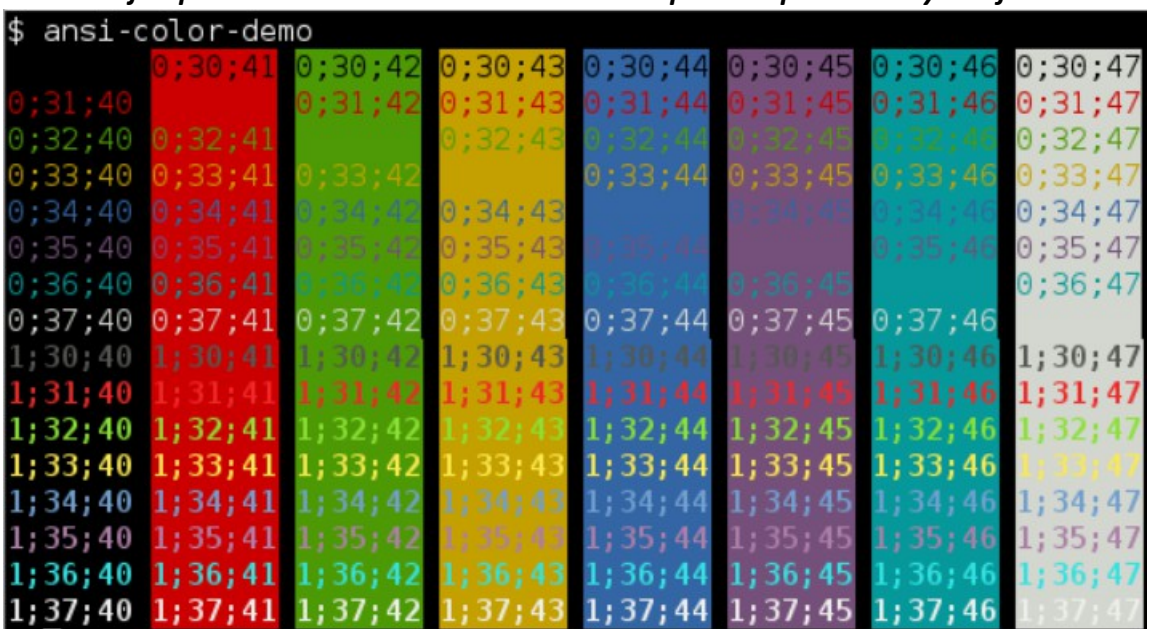
Necesitamos que nuestras aplicaciones para terminales tengan la posibilidad de borrar el contenido del terminal, puedan escribir en cualquier lugar horizontal o vertical sin desplazar el texto que ya hay, puedan cambiar el color de la letra y el fondo de lo que se escriben o pedir... Vas a implementar una clase llamada **MiCo** (abreviatura de Mi Consola que lo implemente) y que lo haga utilizando secuencias de escape ANSI.

El terminal de Windows 10 usa el juego de caracteres 1512 ASCII y no siempre entiende las secuencias de escape UNICODE ni las ANSI. Esto hace que no use el mismo juego de caracteres que su propia interfaz gráfica y que la salida de información de los programas no sea correcta para caracteres acentuados o locales (eñes, etc.). Los terminales compatibles con el estándar ANSI aceptan secuencias de escape (son cadenas de texto) que permiten darles instrucciones. La sintaxis de estas cadenas para controlar el color de la salida es:

`"\033[x;xx;xxm"` donde cada x representa un dígito.

- **El \033 indica el carácter ESC (escape) en octal.**
- **El primer dígito x después del carácter '[' especifica el estilo:**
  - 0 -> Color de texto normal. Ejemplo: texto de color azul: 0;34
  - 1 -> Color de texto Claro . Ejemplo: texto de color azul claro ("1;34")
  - 4 -> Subrayado (no lo implementamos)
  - 5 -> Parpadeo (no lo implementamos)
  - 7 -> Inverso (no lo implementamos)
  - 8 -> Oculto (Pensado para pedir contraseñas al usuario, no lo implementamos)

Ejemplos de valores en secuencias de escape ANSI para color y su efecto



Nosotros no vamos a implementar todas las secuencias de escape, solo algunas. Los códigos para especificar un color (para el texto o para el fondo) utilizan 2 dígitos. A continuación, la lista de códigos para el texto (**para el fondo serían los mismos sumando 10 al color, por ejemplo texto azul es 34 y fondo azul 44**). Para enviar los códigos al terminal, simplemente hay que imprimirlos con print(), println(), etc.

- 0;30 -> Negro 1;30 -> Gris oscuro
- 0;31 -> Rojo 1;31 -> Rojo claro
- 0;32 -> Verde 1;32 -> Verde claro
- 0;33 -> Marrón 1;33 -> Marrón claro (amarillo)
- 0;34 -> Azul 1;34 -> Azul claro
- 0;35 -> Púrpura 1;35 -> Púrpura claro
- 0;36 -> Cyan 1;36 -> Cyan claro
- 0;37 -> Gris 1;37 -> Blanco

Otra secuencia que implementaremos es la de mover el cursor a cualquier lugar:

"\033[LINEA**;**COLUMNA**H"**

Por ejemplo, mover el cursor al principio del terminal: **"\033[0;0H"**

Y el de borrar la pantalla: "\033[2J"

Nota: Como el término de Windows no siempre es compatible con los códigos ANSI vamos a ejecutar nuestros programas en un terminal compatible con ANSI. Junto con la práctica tienes los programas necesarios para 64 bits (ANSICON.EXE, ANSI64.DLL, ANSI32.DLL). Para probar algunos programas, debemos ejecutar este terminal y ejecutar los programas desde él (el programa se puede descargar buscando ANSI v.18.9 en google).

a) (0.8 PUNTOS) Debes crear una clase (pública) llamada **MiCo** que defina constantes estáticas de tipo String con los 8 colores básicos (**TEXTONEGRO, ... , TEXTOGRIS**) y los 8 colores te texto de color claro (**TEXTONEGROCLARO, ... ,TEXTOGRISCLARO**) y los 8 colores de fondo (**FONDONEGRO**, .. , **FONDOGRIS**).

Crea estos métodos estáticos y públicos que debe implementar la clase:

- **cls()** -> borra la pantalla mediante la secuencia de escape ANSI.
- **setColor(String texto, String fondo)** -> debes validar el color que te indican en los argumentos y si no es correcto, no hace nada. Si son correctos, tanto el texto como el fondo, crea la secuencia de escape ANSI y la imprime en el terminal.
- **setCursor(int columna, int fila)** -> posiciona el cursor en la fila y columna del terminal imprimiendo la secuencia de escape ANSI correspondiente.

Nota: Los códigos de escape también pueden escribirse en UNICODE y tienen esta pinta: `"\u001B[104m" para poner color del texto a azul claro.`

b) (0.2 PUNTOS) Prueba la clase **MiCo** en otra clase **U04E03** que tenga un **main()** y realiza varias acciones como borrar la pantalla, escribir varios mensajes donde aparezca tu nombre y apellidos en diferentes colores y posiciones del terminal. Del ejercicio me pasas:

- El código fuente de la clase MiCo.
- El código fuente de la clase U04E03.
- Dos capturas de pantalla de ejecutar U04E03.java, una en Windows (en el terminal ansicon si no te funciona en el de Windows) y otra en GNU/Linux.

EJERCICIO 3. UNA CLASE CON ESTADO NO MODIFICABLE.

Vas a crear una clase llamada **Gna** que encapsule el comportamiento básico de un generador de números aleatorios. ¿Porqué son importantes los números aleatorios? ¿Cómo se generan? Son respuestas que puedes buscar por Intenet, si te interesa.

¿Cómo un ordenador (la máquina más determinista jamás creada por el ser humano) es capaz de generar números aleatorios? Es decir, impredecibles. Bueno, se conocen como pseudoaleatorios y hay varios métodos y casi todos se basan en a partir de una semilla inicial s0 (un valor inicial), ir generando el siguiente s1, a partir de s1 generar s2, ... Además de que los valores devueltos tengan buenas propiedades estadísticas (no sean predecibles) deben de crearse de manera eficiente. Para aplicaciones profesionales es aconsejable además:

- No usar generadores lineales congruenciales (LCG) ni multiplicativos lineales congruenciales (MLCG).
- No usar generadores con períodos (números generados antes de repetir lo que generan) menores de 2^64 o desconocidos.
- No usar los que que produzcan valores donde sus bits menos significativos no sean aleatorios.
- No usar los incorporados en los lenguajes de programación si se necesita cierto nivel de seguridad (por ejemplo Math.random()).

El generador que te propongo que implementes tiene un período de 3.138 x 10^57, y combina la experiencia de eliminación de errores en generadores durante la última década (fuente: Math Recipes in C++ 3Ed). Está pensado para trabajar con enteros positivos de 64 bits para aprovechar el hardware de las actuales CPU (igual debes adaptarlo a los 63 bits de los long en Java). La clase Gna que debes implementar tiene estas características:

Datos encapsulados:

```
u, v, w  // Números de 64 bits
```

Constructores:

- **Constructor por defecto (sin parámetros)**. Sobrecarga el de Object, obtiene el año, el mes, el día, la hora, los minutos y los segundos actuales a partir de la clase **LocalDateTime**, construye una semilla con la fórmula: día * segundos + año * minutos + mes * hora y usa ese valor como la semilla del siguiente constructor.
- Constructor con un número long de parámetro:

```
// ENTRADA: Numero semilla
w = 1;
v = 4101842887655102017L
u = semilla XOR v;
int64();
v = u;
int64();
w = v
int64();
```

- Método oculto al exterior, de uso interno int64() que devuelve un long positivo:

```
FUNCION int64()
        DEFINIR x COMO ENTERO POSITIVO DE 64bits;
        u <- u * 2862933555777941757L + 7046029254386353087L;
        v <- v XOR (v DESPLAZAR_DERECHA 17);
        v <- v DESPLAZAR_IZQUIERDA 31;
        v <- v XOR v DESPLAZAR_DERECHA 8;
        w <- 4294957665 * (w AND 0x7FFFFFFFFFFFFFFFL) + (w DESPLAZAR_DERECHA 32);
        x <- u XOR (u DESPLAZAR_IZQUIERDA 21);
        x = x XOR (x DESPLAZAR_DERECHA 35);
        x <- x XOR (x DESPLAZAR_IZQUIERDA 4);
        DEVOLVER (x + v) XOR w;   // para evitar negativos, pon el bit + significativo a 0
FINFUNCION
```

- Métodos públicos:

```
// Generar aleatorios double 0.0 <= x < 1.0
FUNCION r <- random()
        DEFINIR r COMO DOUBLE;
        DEVOLVER 5.42101086242752217E-20 * int64();

// Generar aleatorio long a <= x <= b
FUNCION r <- valor(long a, long b)
        // Usa int64() y cambia de intervalo y tipo ...;
FINFUNCION

// Generar aleatorio int a <= x <= b
FUNCION r <- valor(entero a, entero b)
        // Usa int64() y cambia de intervalo y tipo ...;

// Generar cadena aleatoria de n caracteres cogidos de la cadena alfabeto
FUNCION r <- valor(entero n, String alfabeto)
        // Si n <= 0 devuelve la cadena vacía
        // Si alfabeto es la cadena vacía usa "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
        // Usa valor(int, int) para generar y devolver una cadena de n letras del alfabeto.
```

Haz la aplicación U0404 que pruebe los métodos de la clase por ejemplo:

```
Construir el Gna1 con Gna(10L) y generar 10 aleatorios enteros entre 0 y 20...

Construir el Gna2 con Gna() generar 20 aleatorios long entre 0 y 20...

Usa Gna1 para generar 10 cadenas aleatorias llamando a valor(10, ""):
   ...
```

EJERCICIO 4. CLASE QUE IMPLEMENTA ARITMÉTICA MODULAR.

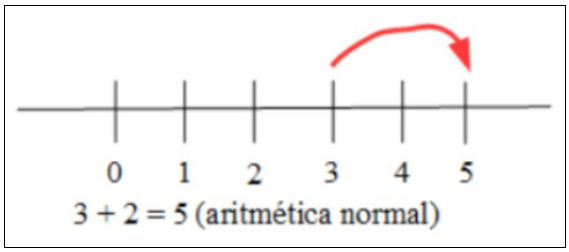
Muchos de los criptosistemas actuales utilizan aritmética modular, algo que estamos muy acostumbrados a usar desde pequeños, aunque nunca lo hemos pensado. Por ejemplo, el sistema horario con formato de 24 horas es hacer operaciones aritméticas usando módulo 24. Las horas posibles son: 0, 1, 2, .. 20, 21, 22 y 23. Si son las 23 horas y le decimos a un amigo "nos vemos dentro de 4 horas", ¿A qué hora quedáis? 23 + 4 (mod 24) = 27 (mod 24) = 3.

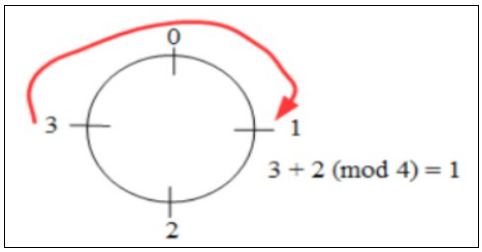
Cuando en un conjunto de números enteros hacemos operaciones aritméticas (+, -, * y /) con aritmética modular p (módulo p), el resultado de las operaciones es siempre otro número entero entre 0 y p-1. Así que configurar un sistema aritmético modular consiste en indicar un número entero positivo **p** que defina el conjunto de valores que pueden devolver nuestras

operaciones y que van desde 0 hasta p-1.

- Escogemos un entero positivo p
- Define un conjunto finito de números {0, 1, 2, 3, ... , p-1} que se denomina Zp.
- Al aplicar operaciones +, -, *, /, etc. el resultado se queda en Zp.

Ahora vamos a ver como funcionan las operaciones aritméticas, y como conseguir que todas estén tan bien definidas como al hacerlas en los números enteros normales.





Operaciones modulares de a y b, ambos pertenecen a Zp, es decir mayor o igual de 0 y menor de p:

$$a + b \pmod p = \begin{cases} a+b & \text{Si } a+b < p \\ (a+b) - p & \text{Si } a+b \geq p \end{cases}$$

$$a - b \pmod p = \begin{cases} a - b & \text{Si } 0 \leq a-b < p \\ p + (a-b) & \text{Si } a-b < 0 \end{cases}$$

$$a \times b \pmod p = \begin{cases} a \times b & \text{Si } a \times b < p \\ (a \times b) \% p & \text{Si } a \times b \geq p \end{cases}$$

Para que la división nos funcione bien, cada valor de Zp debe tener inverso en la multiplicación. El inverso de un número x de Zp sería el valor 1/x de Zp, de forma que x * 1/x (mod p) = 1. Pero esto nos va a dar problemas:

$$a^{-1} = 1/a \pmod p = \begin{cases} b & \text{Si } a \times b = 1 \pmod p \\ \text{PUEDE NO EXISTIR} \end{cases}$$

EJEMPLO 1: ¿1/2 (mod 5)? Vamos a buscar en Z_5 por prueba y error, el entero (de 1 a 4) que al multiplicarlo por 2 (mod 5) nos de 1 (mod 5).

2 x 1 (mod 5) = 2 2 x 2 (mod 5) = 4 2 x 3 (mod 5) = 6 (mod 5) = 1.

Luego 2^{-1} (mod 5) = 1/2 (mod 5) = 3 porque 2 * 3 (mod 5) = 1 (en Z_5 2^{-1} =1/2 es 3).

EJEMPLO 2: ¿1/2 (mod 4)? Lo buscamos por prueba y error en Z_4 = {0, 1, 2, 3}

2 x 0 (mod 4) = 0 2 x 1 (mod 4) = 2 2 x 2 (mod 4) = 0 2 x 3 (mod 4) = 2

Luego 2^{-1} (mod 4) = 1/2 (mod 4) **NO EXISTE!! el 2 no tiene inverso en (mod 4)**.

TEOREMA.

Sea **a** un entero de **Zp**, existe el inverso de **a (mod p)** y se escribe como $\mathbf{a^{-1}}$ o como **1/a** y cumple que **a * 1/a = 1 (mod p)** si y solamente si, el máximo común divisor de a y de p es 1.

Si miras los ejemplos de arriba, mcd(2, 5) = 1 y mcd(4,2) = 2. Por eso en el segundo caso puede haber números a de Z_4 que no tengan inverso (1/a).

> **PREGUNTA:** ¿Hay forma de elegir un p de forma que cualquier **a** de Zp (con $1 \leq a < p-1$) tenga inverso? Si, que **p no comparta divisores comunes con ninguno de ellos**. ¿Qué números no comparten divisores comunes con otros que sean menores? **un primo**. Por tanto, el constructor de tu clase que recibe de argumento el entero que se usa para definir Zp, debe ser capaz de saber si p es primo, para conocer qué operaciones puedes usar con garantías de que funcionen bien. Usa una booleana que indique si p es primo.

Otra de las operaciones que debe implementar cada objeto es calcular el inverso de un número de Zp. Hay varias formas de hacerlo:

- **POR PRUEBA Y ERROR**: pruebo el 1 (mod p), pruebo 2(mod p), ... y pruebo (p-1) (mod p) hasta que obtenga 1 (mod p). Si p es grande → muy ineficiente -> **No lo implementes.**
- **USANDO EL ALGORITMO EXTENDIDO DE EUCLIDES**: De Ecuclides tenemos el algoritmo que calcula el máximo común divisor de dos números mcd(a,b) y el algoritmo extendido mcdExtendido(a,b) que además de calcular el mcd, encuentra dos números u y v que cumplem que au + bv = mcd(a,b). Si llamas a mcdExtendido(a,p), entonces el valor u (mod p) es el inverso de a (mod p).
- **USANDO EL ALGORITMO RÁPIDO DE POTENCIAS Y EL PEQUEÑO TEOREMA DE FERMAT**: ideal **si tienes un algoritmo que calcule potencias rápidamente y p es primo**. Lo implementas porque en otros criptosistemas se puede utilizar, pero no en RSA. :-(

PEQUEÑO TEOREMA DE FERMAT.

Dados **p** primo y un a de Zp, entonces $a^{p-1} = 1 \pmod p$

De esta igualdad, basándonos en que $a^{p-1} = a \times a^{p-2}$ => 1/a (mod p) es a^{p-2} (mod p) y lo podemos calcular con un algoritmo rápido calculando potencias modulares.

- **OPCIÓN ÓPTIMA EL TEOREMA DEL RESTO CHINO**. Se usa en servidores, que tienen que realizarlo muchas veces, pero no lo implementamos porque es más complejo.

Nuestra clase necesita también calcular hashes de mensajes de texto en Zp. La función hash acepta un mensaje de texto y devuelve un número **h** de Zp que es como una huella digital del mensaje porque lo diferencia del resto de posibles mensajes. Las propiedades que debe tener es que cuando cambia el texto, el hash **h** debe cambiar y que sea poco probable que dos textos diferentes generen el mismo hash (aunque es posible, que sea poco probable).

Otra buena funcionlidad que debería tener la clase es la de comprobar si un número entero es primo o no. Es un problema chungo (no polinomial o muy costoso en consumo de CPU si el número es grande). Cuando los números sean grandes hay que descartar hacerlo con los

algoritmos tradicionales porque tardan demasiado. Por eso vamos a cambiarlo por un algoritmo probabilístico que no va a probar todos los casos, pero podemos indicar la certeza con la que queremos saberlo: **esPrimoProbable(n, probabilidad)** donde probabilidad representa un %, un nº del 0 al 1. El algoritmo que usarás usas el **test de Miller-Rabin** y lo aplicaremos un nº determinado de veces al azar hasta alcanzar la probabilidad que se indique. Todos estos algoritmos que necesitas implementar los tienes aquí (de alguno un par de versiones):

```
// ALGORITMO DE EUCLIDES:
// ENTRADAS: 2 enteros a, b
// SALIDA: máximo común divisor de a y b
long mcd(long a, long b)
    SI a < b ENTONCES intercambia a y b; FINSI
    r0 ← a;   r1 ← b;
    MIENTRAS r1 > 0 HAZ
        T ← r1
        R1 ← r0 % r1
        R0 ← t
    FINMIENTRAS
    DEVOLVER r0
FIN
```

```
// ALGORITMO EXTENDIDO DE EUCLIDES MATRICIAL
// ENTRADA: a, b números enteros con a < b.
// SALIDA: r[0] el mcd, [1] u, [2] v
long[] mcdExtendido(long a, long b)
    DECLARA matriz Q = { {1, 0}, {0, 1} }
    MIENTRAS b distinto de 0 HAZ
        q ← a / b
        r ← a mod b
        Q ← matriz { {0, 1}, {1, -q} } * Q
        a ← b
        b ← r
   FINMIENTRAS
   DEVOLVER a, Q[0][0], Q[0][1]
FIN
```

```
// ALGORITMO HASH en Zp
// ENTRADAS: m el mensaje
// SALIDA: un entero con la huella del mensaje
Long simpleHash(string m)
    SI m == null ENTONCES DEVUELVE 0;
    int u = m.longitud - 1
    h ← 0
    PARA i de 0 a m.LONGITUD-1 HAZ
     h ← (h*17 + letra(i)*(letra(u-i)+i)) % p
    FINPARA
    SI h < 0 ENTONCES h = -h
    DEVOLVER h
    SI h < 0 ENTONCES h ← -h;  FINSI;
    DEVOLVER h;
FIN
```

```
// ALGORITMO EXTENDIDO DE EUCLIDES:
// ENTRADA: a, b números enteros con a < b.
// SALIDA: r[0] el mcd, [1] u, [2] v
long[] mcdExtendido(long a, long b)
    DECLARA resultado como un array de 3 long
    long u ← 1, g ← a, x ← 0, y ← b, v;
    MIENTRAS true HAZ
        SI y = 0 ENTONCES
            v ← (g - a * u ) / b
            resultado[0] ← g
            resultado[1] ← u
            resultado[2] ← v
            DEVUELVE resultado
        FINSI
        q ← g / y
        t ← g modulo y
        s ← u - qx
        u ← x
        g ← y
        x ← s
        y ← t
    FINMIENTRAS
FIN
```

```
// ALGORITMO RÁPIDO DE POTENCIAS EN Zp
// ENTRADAS: b base, e exponente y p
// SALIDA: b^e (mod p)
long potenciaMod(long b, long e, long p)
    x ← 1
    b ← b % p
    MIENTRAS e > 0 AND b > 1 HAZ
        SI e % 2 = 1 ENTONCES
           x ← (x * b) % p
           e ← e - 1 // dividir sindecimales
        FINSI
        b ← (b * b) % p
        e ← e / 2
    FINMIENTRAS
    DEVOLVER x
FIN
```

```
// SOBRECARGAR simpleHash
Long simpleHash(int m)
    devolver simplehash( pasar m a String );
FIN

Long simpleHash(long m)
    devolver simplehash( pasar m a String );
FIN

Long simpleHash(double m)
    devolver simplehash( pasar m a String );
FIN
```

```
// ALGORITMO DE PRIMALIDAD PROBABILÍSTICO
long[] primosBajos = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67,
               71, 73, 79, 83, 89, 97, 101 ,103, 107, 109, 113, 127, 131, 137, 139, 149,
               151, 157, 163, 167, 173, 179, 181, 191, 193, 197, 199, 211, 223, 227, 229,
               233, 239, 241, 251, 257, 263, 269, 271, 277, 281, 283, 293, 307, 311, 313,
               317, 331, 337, 347, 349, 353, 359, 367, 373, 379, 383, 389, 397, 401, 409,
               419, 421, 431, 433, 439, 443, 449, 457, 461, 463, 467, 479, 487, 491, 499,
               503, 509};

// ENTRADAS: d (un nº par tal que d x 2^r = n-1 para algún r >= 1) y n (el nº a probar)
// SALIDAS: true si es primo
ALGORITMO millerTest(int d, int n)
    // Elegir un nº aleatorio en [2..n-2] los casos extremos aseguran que n > 4
    int a = 2 + ALEATORIO() % (n - 4);
    int x = potenciaMod(a, d, n);  // a^d % n
    SI x = 1  OR x == n-1 ENTONCES DEVUELVE true; FINSI;
    // Continuar mientras no se cumple algunas de estas cosas
    // (i)   d no sea n-1  (ii)  (x^2) % n no sea 1 // (iii) (x^2) % n no sea n-1
    MIENTRAS d != n-1 HAZ
        x = (x * x) % n;
        d *= 2;
        SI x = 1 ENTONCES DEVOLVER false; FINSI;
        SI x = n-1 ENTONCES DEVOLVER true; FINSI;
    FINMIENTRAS
    DEVOLVER false;
FINALGORITMO

// ENTRADAS: n (el nº a probar) y p (probabilidad de que sea primo)
// SALIDAS: false si con seguridad n no es primo y true si es primo con probabilidad p
ALGORITMO esPrimoProbable(int n, int p)
    SI n <= 1 OR n = 4  ENTONCES DEVUELVE false; FINSI;  // Casos extremos
    SI n <= 3 ENTONCES DEVUELVE true; FINSI;
    SI n <= primosBajos[primosBajos.length-1]
    ENTONCES
        SI x está en la lista de primosBajos DEVUELVE true SINO DEVUELVE false FINSI
    FINSI
    int K ← entero superior( log(1 – p) / log(0.5) );
    // Encuentra r tal que n = 2^d * r + 1 para algún r r >= 1
    int d = n - 1;
    MIENTRAS d % 2 = 0 HAZ d /= 2; FINMIENTRAS;
    PARA i = 0 HASTA k-1 HAZ
         SI !millerTest(d, n) ENTONCES DEVUELVE false; FINSI;
    FINPARA
    DEVUELVE true;
FIN
```

Implementa la clase Zp63 que permita definir objetos que puedan hacer operaciones con **números enteros positivos de hasta 63 bits**:

- Estado:
  - El módulo p en el que se hacen las operaciones.
  - Booleana que indique si el p es primo.
- Comportamiento:
  - Constructor que acepte el parámetro p y que tenga máximo 63 bits. Precondiciones: mayor de 1L
  - Getters de sus datos.
  - Suma, resta, multiplicación y división en módulo p (operaciones externas) que aceptan dos datos de tipo long y devuelven otro long.

- Método a nivel de clase que diga si el parámetro n es primo: **boolean esPrimoProbable(long n, int prob)** (puede usar el algoritmo probabilístico con probabilidad prob).
- Algoritmo de exponenciación modular rápido para calcular potencias modulares: **potenciaModular(long base, long exponente)**.
- Inverso modular de a, basado en el pequeño teor. de Fermat. De uso interno **inversoFermat(long a)**
- Máximo común divisor **mcd(long a, long b)** a nivel de clase.
- Versión extendida del mcd (una de las dos, la que quieras) a nivel de clase también **mcdExt(long a, long b)**.
- Método long **inversoModular(long a)** que calcule el inverso en Zp de un número a y que use el teor. de Fermat si p es primo o el algoritmo extendido de Euclides si no lo es. Convierte antes a en un número de Zp (entre 0 y p-1 ambos incluidos). Si no existe, devuelve -1.

EJERCICIO 5. RECURSIVIDAD Y BACKTRACKING.

Hay un gran número de problemas cuya solución requiere la aplicación de métodos de tanteo de forma sistemática, es decir, no hay una fórmula/algoritmo tradicional que te de el resultado, hay que ir probando todas las opciones posibles. La técnica de **backtracking** divide la solución de un problema en pasos y define la solución del paso i-ésimo en función de la solución del paso i-ésimo + 1.

inicio -> pruebo paso1 -> pruebo paso2 -> ... -> pruebo paso i-ésimo -> pruebo paso i-ésimo + 1

Si en cierto paso se detecta que no es posible llegar a una solución satisfactoria, se retrocede (**back**) a un paso anterior para probar con otra posible opción pendiente de explorar (**tracking**, porque en cada paso se recuerdan las posibilidades pendientes de probar).

La recursividad nos sirve en estos problemas para tener memoria (tracking) en cada paso de forma sencilla. Haz la clase pública **Laberinto** donde defines:

- Una constante de la clase llamada **PARED** que sea el carácter '*'
- Una constante de la clase llamada **CAMINO** que sea el carácter '.'
- Una constante de la clase llamada **ENTRADA** que sea el carácter 'E'
- Una constante de la clase llamada **META** que sea el carácter 'M'
- Una variable miembro **laberinto** como un array bidimensional de caracteres.
- Una variable miembro **ruta** como un string vacío.
- Un objeto **Gna()**.
- Unas coordenadas **posX**, **posY** donde se comienza a imprimir el laberinto.

En la clase ejecutable **U04E05**, en el método main() de la aplicación:

Pregunta y valida **(mayor de 3)** el nº de filas del laberinto **fL**
Pregunta y valida **(mayor de 3)** el nº de columnas del laberinto **cL**
Pregunta y valida **(0 < p < 1)** la probabilidad **p** de que una celda tenga pared
Pregunta y valida **(0 <= fS < fL-1)** la fila **(fS)** donde está la salida
Pregunta y valida **(0 <= fM < fL-1)** la fila **(fM)** donde está la meta

Pregunta por una semilla **s** para el Gna del laberinto.

A continuación, crea un objeto de la clase Laberinto pasando los argumentos leídos **fL**, **cL**, **fE**, **fM**, **semilla, p.**

- **fL**: entero filas del laberinto.
- **cL**: entero columnas del laberinto.
- **fE**: entero fila donde está la entrada al laberinto. La columna siempre será la 0.
- **fM**: entero fila donde está la meta. La columna siempre será la última columna del laberinto.
- **semilla**: es la semilla usada para el Gna del laberinto. Es mejor usar nuestro Gna, porque podemos ir repitiendo la secuencia de números obtenida (si damos la misma semilla obtenemos la misma secuencia) mientras hacemos que el programa funciones.
- **p:** probabilidad de que haya pared en una celda que no sea del borde (siempre pared).

El constructor de Laberinto crea y rellena el array laberinto (array de caracteres bidimensional de tamaño **fL x cL**). El constructor a continuación:

- Rellena el array del laberinto con espacios en blanco pero las celdas que estén en el borde del rectángulo se quedan como **PARED**.
- Pone la entrada en la fila fE y la columna 0.
- Pone la meta en la fila fM y en la columna cL-1.
- Crea el Gna que usará inicializándolo con el argumento semilla.
- Recorre las celdas que no estén en los bordes (ni cerca de la salida o la meta, mira la nota) y para cada una:

  x ← nº aleatorio de Gna, 0.0 <= x < 1.
  SI x < p **ENTONCES** celda ← PARED

Nota: Cuando la salida o la meta están bloqueadas y no hay forma de llegar a ellas no tiene gracia, así que es preferible dejar sin pared las zonas alejadas de los bordes y cercanas a la entrada y la salida.

Crea el método público **Laberinto.dibuja(int fila, int columna)** que fije posX a fila y posY a columna, y utilizando la clase **MiCo** se posicione en las coordenadas (x,y) del terminal y dibuje el laberinto a partir de (fila, columna) con estos colores:

- En color gris claro el fondo y azul la pared.
- La SALIDA y la META en color rojo.

La clase Laberinto debe proporcionar un método **boolean moverA(int x, int y)** que va a buscar un camino hasta la meta desde la posición (x,y) en el laberinto (si la hay) usando backtracking: ir probando movimientos y si se da cuenta de que ha llegado a un callejón sin salida, deshace lo realizado, vuelta atrás y continúa en un punto anterior. Utiliza la recursividad para realizar la vuelta atrás.

El programa principal asume que inicialmente se está en la entrada (fE,0) y hará una primera llamada al método **moverA(fE, 1)**, que intenta llegar a la meta desde (fE, 1). La función comprueba si ya ha llegado (caso trivial) y devuelve true. También comprueba si ha llegado a un lugar imposible (pared, fuera del laberinto, entra de nuevo en el camino que está explorando ahora mismo, etc.). Si es un lugar imposible sale devolviendo false.

Si aún no ha llegado a la meta, pero es un paso posible de realizar, intenta de forma recursiva acercarse de nuevo a la meta realizando movimientos al norte ('N' = bajar de fila), al este ('E' = aumentar columna), al sur ('S' = aumentar fila) y al oeste ('O' = disminuir la columna). Si ese movimiento ha tenido éxito, lo anota en la variable ruta y lo indica a quien la llamó. Pero si en una posición, después de haber agotado todas las operaciones posibles, no ha tenido éxito, hace backtracking e informa a quien la llamó que esa ruta no tiene futuro. El algoritmo:

```
boolean moverA(int f, int c)
    SI laberinto[f][c] == META ENTONCES RETURN true; FINSI
    SI laberinto[f][c] == PARED OR laberinto[f,c] == CAMINO OR laberinto[f][c] == ENTRADA
    ENTONCES
        RETURN false;
    FINSI
    laberinto[f][c] = CAMINO
    // Dibujar CAMINO también en consola...
    boleana intento;            // Desde aquí intentamos otro paso
    // Intentar ir al NORTE...
    intento = moverA(f-1, c)
    SI intento ENTONCES
        ruta= 'N' + ruta
        RETURN true;
    FINSI
    // Intentar ir al este…
    // Intentar ir al sur…
    // Intentar ir al oeste…
    // Si llega aquí, ha agotado posibilidades sin éxito -> se descarta este camino
    laberinto[f][c] = ' ';
    // Dibujar el espacio en la consola...
    RETURN false;  // Este es el backtracking
```

Con este código igual tienes algunos problemas de salida de índices. Si te ocurre, aprovecha la evaluación en cortocircuito de las expresiones lógicas para corregirlo.

Figura 1: encuentra la meta en un laberinto.

Nota: Cuando la meta está bloqueada y no hay forma de llegar a ella, o el camino atraviesa una zona amplia y sin continuidad, **este algoritmo es ineficiente** porque intenta explorar todas las posibilidades desde cada posición en la que está y pierde tiempo explorando varias veces las mismas rutas inútiles y acaba en ellas en diferentes intentos aunque evita entrar en las posiciones que ahora mismo está explorando (el camino actual), cuando retrocede y deshace camino (backtracking), puede volver otra vez a la misma posición inútil (f,c) que ya exploró en otro camino anterior -> esto se traduce en muchas exploraciones inútiles.

EJERCICIOS JEDI (EL QUE TE TOQUE)

Como siempre, aunque solo tenéis que entregar el que os toca, deberíais intentar hacerlos todos para practicar.

EJERCICIO 6. CLASES CON MÉTODOS Y PRECONDICIONES.

Crea una clase pública **U04E06** que pruebe todos los métodos de otra clase llamada **EstaUni** (Estadística Unidimensional) y defines en esta última métodos destinados a realizar operaciones estadísticas sobre una lista de datos. Los métodos y datos serán de los objetos.

Datos:

- Un array de doubles con los valores. Tendrá getter y setter y operaciones de uso externo nuevoDato(double), borrarDato(int p) y cambiarDato(int p, valor). Piensa en las consecuencias de cada operación para el funcionamiento de la clase.
- Un double con la media de los valores. De uso interno.
- Un booleano que indique si la media está calculada ya (para ahorrar tiempo y no tener que calcularla innecesariamente). De uso interno.
- Un booleano que indique si los datos están ordenados actualmente de memor a mayor o no. De uso interno.
- int que indica cantidad de decimales con la que mostrar datos. Tiene getter y setter.

Constructores:

- Uno que acepte el array de valores y la cantidad de decimales de los resultados y recuerde que la media no se ha calculado aún y que los datos no se han ordenado aún.
- Otro que acepte el array de valores y llamando al anterior, fije a 3 el nº de decimales.

Métodos:

- Uno de uso interno llamado redondea que acepte un double y devuelva otro double con con el parámetro redondeado al número de decimales que tenga el objeto.
- toString () para poder imprimir los valores con el nº de decimales fijados en el objeto con el formato: "[valor_1, valor_2, ..., valor_n]".
- Implementa las operaciones que se muestran a continuación y utilizando Aserciones (al menos una vez) y excepciones (al menos una vez) implementa sus precondiciones:

media(): resume todos los datos en un único valor cuantitativo que los representa. Debe haber al menos 1 dato. La fórmula se ve a la derecha:

$$\bar{x} = \frac{\sum_{i=1}^{n} x_i}{n}$$

mediana(): **Debe haber al menos 2 datos**. valor que deja a la mitad de valores por debajo y a la mitad por encima. Si hay un nº de datos impar, la mediana es el dato que al ordenarlos deja la mitad por debajo y la otra mitad por encima (dato en la posición (n-1)/2. Si hay un nº par de datos (n es par), la mediana es la media de los datos que haya en las posiciones (n-2)/2 y n/2. Debe haber al menos 2 datos.

percentil(p): **p puede ir del 1 al 99 y debe haber al menos 2 datos.** Si Pk es el percentil k, es el valor que deja el k% de las datos ordenados de menor a mayor, por debajo del valor Pk. El percentil 50 es la mediana. Hay que devolver:

```
pos ← entero(n x p/100)
Si pos tiene decimales → devolver dato[pos]
sino
   si pos >= 1 entonces pos1 <- pos-1  sino pos1<- pos;
   devolver la media del dato[pos] y dato[pos1]
finsi
```

varianza(): σ^2 si los datos son de una población de media $\overline{x}$.

$$\sigma^2 = \frac{\sum_{I=1}^{n} (x_i - \overline{x})^2}{n}$$

varianzaM(): varianza si los datos son de una muestra (una parte) de la población. La fórmula a la derecha:

$$\sigma^2 = \frac{\sum_{I=1}^{n} (x_i - \overline{x})^2}{(n-1)}$$

desvEstandar(): desviación estándar de los datos de una población.

$$\sigma = \sqrt{varianza()}$$

desvEstandarM(): desviación estándar de una muestra.

$$\sigma = \sqrt{varianzaM()}$$

curtosis(): mide lo escarpado o rugoso de la distribución de n datos. Si es positivo, los datos están muy agrupados en torno a la media, empinada. Si es negativo, los datos están muy repartidos y la curva es achatada. Si es 0, los datos siguen una distribución normal. En la fórmula, s es la desviación estándar y la $\overline{x}$ es la media.

$$curtosis = \sum_{i=1}^{n} \frac{(x_i - \overline{x})^4}{n\, s^4}$$

CAFisher(): coeficiente de asimetría de Fisher. Mide si los datos son simétricos respecto a la media. Si < 0, más datos por encima de la media. Si > 0, más datos a la izquierda. Si = 0, simétrica. $\overline{x}$ es la media y σ es la desviación estándar.

$$C_{AF} = \frac{\sum_{i=1}^{n} (x_i - \overline{x})^3}{n\,\sigma^3}$$

rango(): devuelve la anchura del rango de valores que tienen los datos, es decir, el máximo menos el mínimo valor.

normalizar(): Genera y devuelve un nuevo EstaUni con datos (un nuevo array), que tiene de media 0 y de desviación típica 1. Son datos que tienen las mismas propiedades que los originales, pero están concentrados en valores más reducidos. Esto permite comparar datos que no coinciden en los valores que toman (por ejemplo, qué influye más en el precio de una casa, el número de baños, los metros cuadrados, ...).

Devuelve un array de n elementos Zi, donde cada $Zi = \frac{Xi - media()}{desvEstandar()}$

escalar(a,b): normaliza los datos a cualquier intervalo [a,b]. Genera un nuevo EstaUni de n elementos con una versión escalada de los datos X al intervalo [a,b], donde cada elemento Ei se define como:

$$Ei = a + \frac{(Xi - min(X)) * (b - a)}{max(X) - min(X)}$$

b) (0.2 PUNTOS) Además del código de la clase, entrega captura de ejecutar un programa que lo pruebe con el segundo conjunto de datos (te paso el primero):

- Primer conjunto: Notas de la parte teórica del primer control de DAM:
  `5.60, 0.00, 6.53, 5.63, 6.73, 7.88, 6.36, 5.91, 0.00`
- **Segundo conjunto:** Notas de la parte teórica del segundo control de DAM:
  `5.01, 5.44, 4.10, 7.30, 6.17, 4.94, 4.62`

Te paso el resultado de la ejecución del primero para que te sirva de guía:

```
  ESTADISTICA UNIDIMENSIONAL
  ==========================

1. Definir datos desde teclado.
2. Parametros.
3. Normalizar y escalar.
4. Salir.

  Escoja una opcion (1-4):
```

```
  Escoja una opcion (1-4): 1
Cuantos decimales quiere en los resultados? 2
Cuantos datos quiere definir? 9
  dato[0]: 5,6
  dato[1]: 0
  dato[2]: 6,53
  dato[3]: 5,63
  dato[4]: 6,74
  dato[5]: 7,88
  dato[6]: 6,36
  dato[7]: 5,91
  dato[8]: 0
```

```
  Escoja una opcion (1-4): 2
Parametros centrales
  Media: 4.96 Mediana: 5.91
  Percentiles P1: 0.0 P25: 5.6 P50: 5.91 P75: 6.53 P99: 7.88
Parametros de dispersión
  Varianza: 7.45 Varianza Muestral: 8.39 Desviación Estándar: 2.73 Rango: 7.88
Parametros de forma
  Curtosis: 2.61 -> Empinada
  Coeficiente de Asimetría de Fisher: -1.12 -> A la izquierda
```

```
  Escoja una opcion (1-4): 3
Datos normalizados: [ -1.817, -1.817, 0.234, 0.245, 0.348, 0.512, 0.575, 0.652, 1.069 ]
Datos escalados a [0,1]: [ 0.0, 0.0, 0.711, 0.714, 0.75, 0.807, 0.829, 0.855, 1.0 ]
```

EJERCICIO 7. TEST DE ALEATORIEDAD.

Trabajamos en un fábrica de dados y queremos saber si un dado que hemos fabricado está bien o mal fabricado. Necesitamos una clase que nos compruebe si los resultados que genera son aceptablemente aleatorios o no lo son. Así que tenemos el problema de saber si algo es aleatorio, por ejemplo, si generamos 1 millón de números aleatorios entre 0.0 y 1 (excluido). Pero, ante una secuencia de números ¿Cómo saber si es o no aleatoria? O al menos, ¿Cómo de aleatoria es? ¿Hay alguna comprobación que podamos realizar para saberlo? Las únicas pruebas que podríamos hacer a un conjunto de valores son pruebas estadísticas, para intentar averiguar si siguen algún patrón, en cuyo caso no serían valores aleatorios.
Una prueba sencilla se denomina "bucket" y consiste en definir una serie de recipientes "cubetas" y repartir todos los valores en ellas, en cada una vamos contando cuantos valores caen. Por ejemplo, si generamos números enteros de 1 a 6, podríamos definir 6 cubetas. Si generamos 1000 valores aleatorios, podemos esperar que haya aproximadamente la sexta parte de los valores en cada cubeta. Bueno, en realidad al ser aleatorios tampoco tendría que ser exactamente la sexta parte, igual tener algunos más o menos en alguna es aceptable para algunos usos, quizás para otros no. Debes hacer una clase llamada **TestAleatorios** que nos ayude a comprobar esto implementando un array con datos de tipo double y 2 métodos de uso externo, donde cada uno implementa uno de estos tests:

TEST DE UNIFORMIDAD ¿LOS VALORES ESTÁN ALEATORIAMENTE REPARTIDOS?

Clasificamos los números aleatorios en un número de cubetas, contamos los que caen en cada cubeta, calculamos la probabilidad teórica de cada cubeta, a partir de ella se calcula el coeficiente chi cuadrado con tantos grados de libertad como número de cubetas menos 1 hayamos usado. Un ejemplo a una serie de números entre [0.0, 1).

Primero calculamos la probabilidad de la realidad. Si hacemos 100 cubetas, cada cubeta se queda con los números que caen dentro de un trozo del ancho de posibles valores:

rango = (max – min) / nºCubetas = (1.0 – 0.0)/100 = 0.01, así que:

- La cubeta 0 recoge los números entre 0 y 0.01.
- La cubeta 1 recoge los que estén por encima o igual a 0.01 y menores que 0.02.
- etc.

Ahora calculamos la probabilidad de la teoría. La probabilidad de que un número caiga en una cubeta es de 1/100 (si es uniforme), es decir 0.01. Por tanto, el valor esperado de números en cada cubeta será de: números generados * 1 / nºcubetas = 1000000 * 0.01 = 10000

Los grados de libertad serían el nº de resultados posibles menos 1, en este caso el nº de cubetas menos uno, es decir 100 - 1 = 99.

Con estos valores se calcula el coeficiente chi cuadrdado. Miramos en una tabla de chi cuadrado el % en el que aparece este coeficiente, y si para 5% es mayor y para 95% es menor (si está entre el 5% y el 95%) damos por pasado el test de uniformidad (los valores no se concentran en ciertos valores más de lo que nuestro concepto de azar permite). Si no se da esa condición, no pasa el test.

Test Chi-Cuadrado (χ^2) explicado con otro ejemplo

Imagina que hay un par de monedas y tenemos la sospecha de que pueden estar trucadas. ¿Cómo probarlo? Lanzamos las monedas 100 veces y anotamos los resultados de los lanzamientos en una tabla (se llama tabla de contingencias).

100 lanzamientos	2 caras	Cara y cruz	Dos cruces
Observado (realidad)	28	51	21
Probabilidad	0,25 (1/4)	0,5 (1/2)	0,25 (1/4)
Esperado (teórico)	25	50	25

Bien, la realidad no coincide con la teoría ¿Eso significa que las monedas están trucadas? ¿Tienen tendencia a salir cara? ¿O esta diferencia es razonable al ser un experimento aleatorio? Vamos a calcular el coeficiente chi cuadrado:

$$\chi^2 = \sum_{i=1}^{k} \frac{(E_i - O_i)^2}{E_i}$$

Donde

- k son los posibles resultados (en este caso 3: 2caras, cara y cruz y 2 cruces)
- Oi son los valores observados (la realidad).
- Ei son los valores esperados (la teoría).

En nuestro experimento (lanzar dos monedas) hay 3 posibles resultados (2caras, cara+cruz y 2cruces) por tanto i va de 1 a 3.

$$\chi^2 = \frac{(28-25)^2}{25} + \frac{(51-50)^2}{50} + \frac{(21-25)^2}{25} = \frac{3^2}{25} + \frac{1}{50} + \frac{4^2}{25} = \frac{9}{25} + \frac{1}{50} + \frac{16}{25} = 1.02$$

Aquí vemos un trozo de la tabla χ^2 que es sencilla de entender cuando te la explican, claro.

	1%	5%	95%	99%
$\nu = 1$	0.000157	0.00393	3.84	6.63
$\nu = 2$	0.0201	0.103	5.99	9.21
$\nu = 3$	0.115	0.352	7.81	11.3
$\nu = 4$	0.297	0.711	9.49	13.3
$\nu = 5$	0.554	1.15	11.1	15.1
$\nu = 6$	0.872	1.64	12.6	16.8

Los valores se agrupan en filas según los **grados de libertad** (primera columna). Los grados de libertad es un número entero cuyo valor es uno menos que el número de posibles resultados de nuestro experimento (k-1). Como tenemos 3 posibles resultados (2caras, cara+cruz y 2cruces) los grados de libertad son 3 - 1 = 2.

Así que nos tenemos que fijar en la fila que tenga 2 grados de libertad (recuadro en rojo). Si miramos el valor de la columna 1% (0.0201), debemos interpretarlo de la siguiente manera: "El valor que hemos calculado del coeficiente χ^2 debería ser menor de 0.0201 solo el 1% de las veces. Es decir, si hemos repetido el experimento 100 veces, solamente una vez podemos

tener un χ^2 menor de 0.0201. Si encontramos que varias veces tenemos un valor inferior, podríamos concluir que las monedas están trucadas. Una interpretación similar se puede hacer para la columna del 5%.

Si nos vamos a la columna 95%, "el valor de χ^2 debería ser menor de 5.99 el 95% de las veces" o "debería ser mayor de 5.99 solamente el 5% de las veces". Similar para el 99%.

Nuestro χ^2 está entre el 5% y el 95%, así que no podemos deducir que estén trucadas. Sin embargo, si al repetir el experimento hubiésemos calculado un χ^2 de 10 por ejemplo, esto no debería de ocurrir más que el 1% de las veces (porque está por encima del 99%). Y es un indicador de que las monedas están trucadas.

En general, los profesionales suelen fijarse en el 5% y el 95% para aceptar o descartar lo aleatorio que es un experimento. Si es menor del valor 5% o mayor del 95%, se rechaza por ser poco aleatorio.

Dejamos las monedas y vamos a aplicar el test a las secuencias de valores. Implementa un método de la clase TestAleatorio llamado uniformidad(int k) y otro sobrecargado a nivel de clase uniformidad(int k, int[] observados, double prob[])

El primero acepta el número de cubetas que quieres emplear (k) y asume que la probabilidad teórica es la misma en todas las cubetas. Luego calcula el máximo y el mínimo valor de los datos del objeto para saber lo grande que es cada cubeta y contar los valores que han caído en cada una,. Luego calcula cuántos en teoría debían haber caído. Muestra la tabla de contingencia (resultados reales y teóricos) y calcula el parámetro xi cuadrado de los datos mostramndo también los grados de libertad.

La segunda versión, es a nivel de clase porque no usa los datos del objeto y está pensado para problemas como el del ejemplo. Indicamos el número de cubetas en el primer parámetro k, en el array observados (que debe tener k enteros, uno por cubeta) pasamos los que han caido en cada cubeta en realidad, y en el argumento prob[] pasamos la probabilidad teórica de cada cubeta. El método debe calcular cuántos le corresponden en teoría, calcular y mostrar la tabla de contingencia y calcular y mostrar el paráemtro xi cuadrado y los grados de libertad.

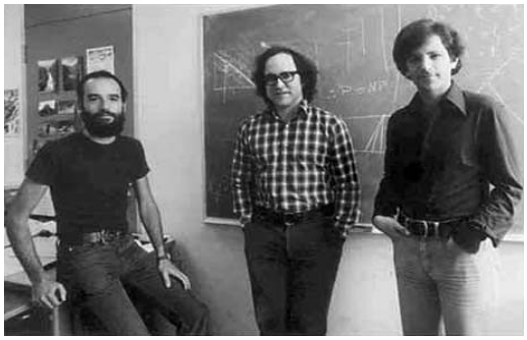
```
// Llamada para mostrar resultados del ejemplo
TestAleatorio.uniformidad( 3,new int[] {28, 51, 21 }, new double[]{0.25, 0.50, 0.25} );
```

EJERCICIO 8. CRIPTOSISTEMA ASIMÉTRICO RSA.

Los criptosistemas simétricos utilizan la misma clave para encriptar (cerrar o proteger la información) y para desencriptar (abrir o hacer accesible la información). Es como la llave de una caja fuerte, la misma llave abre o cierra la puerta. Los asimétricos sin embargo, utilizan dos llaves diferentes, una **solo sirve para cerrar y se llama clave pública** porque no pierdes seguridad si regalas copias a desconocidos (aunque sean cacos malotes), solo la podrán usarla para cerrar. Y una clave distinta llamada **clave privada, que sirve para abrir**. Esta es importante que solo la tenga el dueño de lo que se protege. Solo él puede acceder (abrir).

¿Cómo funcionan, en qué se basan estos criptosistemas? Utilizan **problemas matemáticos**

denominados computacionalmente intratables, por que incluso el ordenador más potente jamás construido, ejecutando el mejor algoritmo que resuelva ese problema, tardaría más tiempo en solucionarlo sin conocer la clave privada, que edad tiene el universo en que vivimos.



Calcular los factores primos de un número entero N consiste en calcular los números primos que al multiplicarlos entre sí generan al número N. Ejemplo: Factoriza el número 40:

40/2 = 20
20/2 = 10
10/2 = 5
5/5 = 1
Luego 40 = 2 x 2 x 2 x 5

¿Y este es el problema difícil? Bueno, pues sí, siempre que se coja un número bastante grande (con 64bits tienes números de unos 18 dígitos decimales) y si encima tiene pocos factores primos (un par de ellos), encontrarlos puede ser muy chungo. Pero si coges números de 1024 bits y a mala leche, ni te cuento, de hecho es un problema de orden exponencial, es decir, actualmente intratable de forma computacional.

Creado por Rivest, Shamir y Adleman, fue la primera implementación pública de un sistema de clave asimétrica como propuso Diffie y Hellmam. Es el único que aún se mantiene con un orden exponencial muy potente. Patentado en 1983 por PKP de EEUU. Debes implementar una clase llamada **SimpleRSA** que usando la clase **Zp** del ejercicio anterior implemente su funcionamiento, que te describo a continuación:

CONFIGURACIÓN DE UN SISTEMA RSA con claves de N bits.

PASO 1. Se eligen al azar **dos números primos p y q** de aproximadamente N/2 bits cada uno y se calcula **n = p * q**. Hace cálculos en Zn. A partir de n se calcula otro valor llamado **orden de n (φ(n)) que es:** φ(n) = **(p-1) * (q-1)** que es sencillo de calcular si conoces p y q.

PASO 2. Se selecciona un entero positivo e, $1 \leq e < \varphi(n)$ de modo que sea primo con el orden del grupo, es decir, el máximo común divisor de e y φ(n) es 1. Siempre suele ser el mismo valor en todos los RSA, el cuarto número de Fermat, $F_4 = 2^{16} + 1 = 65.537$. Puedes definirlo como una constante o dejar que lo indiquen, como quieras.

PASO 3. Mediante el algoritmo de Euclides extendido calcula el inverso de **e** en **Zorden**, al que llamamos **d = 1/e (mod** φ(n)). Se cumple que e x d = 1 (mod φ(n)) y $1 \leq d < \varphi(n)$.

PASO 4. La **clave pública del usuario U es la pareja de números (n,e)** y **la clave privada es el número d** aunque debe mantener en secreto p, q y φ(n) a partir de los que se obtiene d.

Un criptosistema RSA se puede usar para encriptar, desencriptar, intercambio seguro de claves de sesión y firma digital. Tu vas a implementar la configuración, la encriptación y desencriptación, la firma de un mensaje y la validación de la firma. Elementos necesarios para implementar esta funcionalidad es tener estos 3 métodos auxiliares en la clase:

ELEGIR TAMAÑOS DE BLOQUE, CODIFICAR Y DECODIFICAR.

RSA es un cifrador de bloques, si lo aplicamos solo a texto, debemos conocer el alfabeto que usan nuestros mensajes para saber cuántas letras debe tener cada trozo (bloque) de mensaje con el que trabaja RSA. Para encriptar un mensaje, debemos romperlo en bloques, codificar cada uno en un número de Zn, que RSA use el número para generar otro número, decodificar el número que devuelve en un mensaje del tamaño adecuado (rellenando con el primer carácter del alfabeto si le faltan letras para alcanzar el tamaño del bloque).

Bloque de letras --> codificar a nº de Zn --> RSA --> nº de Zn ---> decodificar a bloque

¿Cómo calcular el tamaño de los bloques? (cuantas letras forman un bloque) Debe ser la cantidad de letras que al multiplicarlas no supere al n. Si tengo un alfabeto de 26 letras:

```
tama <- 1
peso <- longitud del alfabeto
MIENTRAS peso < n HAZ
       peso <- peso * 26
       tama <- tama + 1
FINMIENTRAS
```

¿Cómo codificar? Del mensaje ve cogiendo letras, quitando las que no estén en el alfabeto, hasta tener un nº de letras igual al tamaño de bloque. El alfabeto es como tener un sistema de numeración con base el nº de letras del alfabeto (lo llamamos **base a**), lo que debes hacer es convertir el bloque del mensaje a decimal:

Bloque de letras del mensaje = Cn ... C1 C0
Número en decimal = C0 x 1 + C1 x a + C2 x a^2 + … + Cn x a^(n-1) < n

¿Cómo decodificar? Una vez que RSA devuelve un nº, hay que volver a generar un bloque de mensaje del tamaño que use RSA. Transformar el nº decimal en otro escrito en base a (las letras del alfabeto), igual que si pasaras a base 2, 8 o 16, etc. Ir haciendo divisiones sucesivas entre la base hasta quedarte con un número inferior a la base a, e ir generando la letra de la posición que indica el resto y el útimo cociente. Rellena con el primer carácter del alfabeto (representa el 0 y será como añadir ceros por la izquierda) hasta completar la longitud que debe tener. Ejemplo: A quiere enviar un mensaje m cifrado a B, le pide su clave pública y:

ENCRIPTAR m.

- Primero A codifica el mensaje M en elementos de Z/nZ (ya lo hemos visto).
- Luego utiliza la clave pública de B que es (n_b, e_b) para calcular

$$m^{e_b}(mod\, n_b)=c$$

DESENCRIPTAR m.

- B calcula:

$$c^{d_b}(mod\, n_b)=(m^{e_b})^{d_b}(mod\, n_b)=m$$

Ejemplo: Si el usuario A quiere enviar el mensaje M al usuario B y el alfabeto = {A..Z} sin la Ñ (26 letras codificadas del 0 .. 25). B ha configurado su RSA con p = 281 y q = 167, n = 46927 y φ(n)= 46480. B elige el e = 39423 y se asegura que el mcd(39423, 46480) = 1. Ahora calcula el

inverso 1/39423 (mod 46480) = 26767 = d. La clave pública de B es la pareja de números 46927 y 39423 y su privada 26767.

Debemos codificar bloques de un tamaño de 4 letras (porque 26 x 26 x 26 < φ(n)).

A codifica el mensaje m = "YES" → se codifica en Y x 26 x 26 + E x 26 + S = 16346 (si el mensaje es más corto añadimos A por la izquierda (es el cero con ese alfabeto) hasta alcanzar los 4 caracteres) -> "AYES" sería el mismo valor.

A pide la clave pública a B y la usa para encriptar el mensaje m para obtener la cifra C:

$C = 16346^{39423} \pmod{46927} = 21166$

A decodifica la cifra:

cifra % 26 -> "C",
(cifra/26) % 26 = "I"
(cifra/26/26) % 26 = "F"
(cifra/26/26/26) % 26 = "B"

Envía la cifra "BFIC" (si tuviese menos de 4 letras completaría con A por la izquierda hasta llegar a 4).

B recibe "BFIC" y lo codifica en 21166.

Recupera m usando su privada, calculando $21166^{26767} \pmod{46927} = 16346$
Decodifica 16346 y obtiene YES.

Debes implementar en la clase:

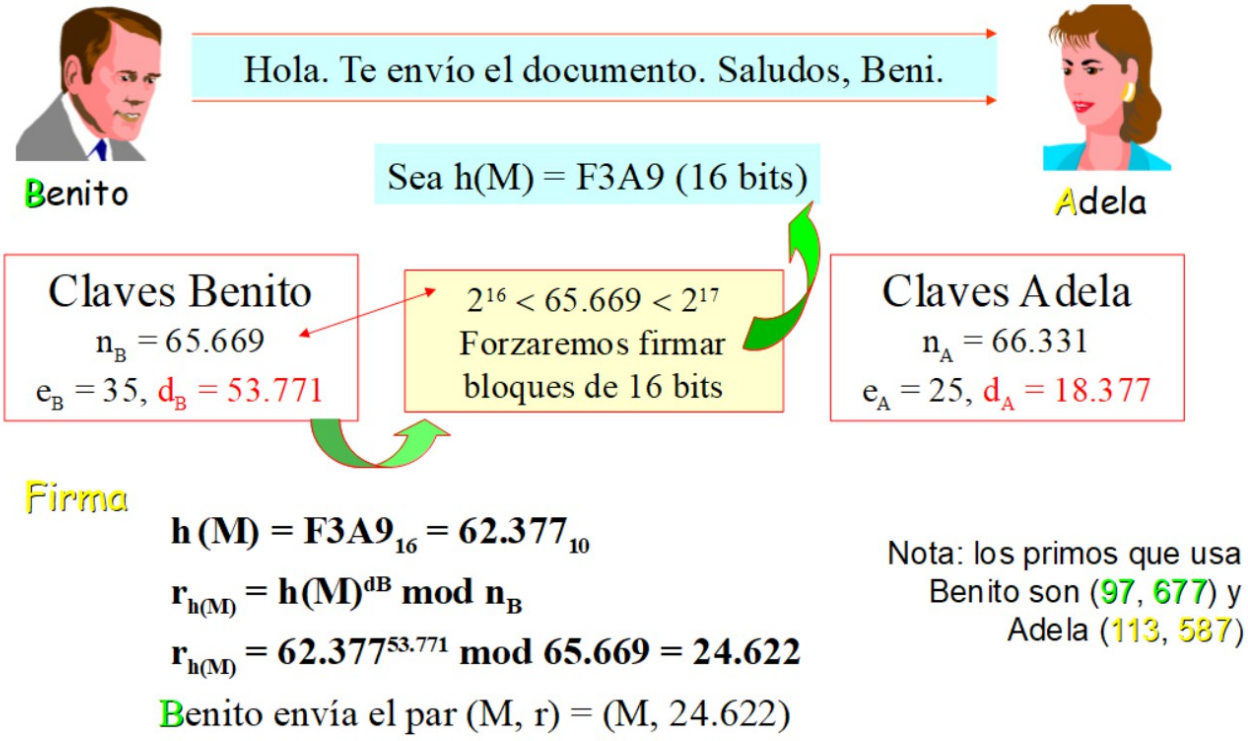
- **Constructor con un parámetro int:** indica el número de bits n que debe usar el criptosistema. Busca primos probables p y q de unos n/2 bits cada uno y calcula n, orden, e es el 4º nº de Fermat, y calcula d (la privada, 1/e mod(orden)).
- **Constructor con 3 parámetros long: p, q y e.** Si p y q no son primos lanza excepción de parámetros ilegales. En otro caso, calcula n, orden y d.
- **String encriptar(String m, long nb, long eb, String alfabeto):** encriptas un mensaje para enviarlo protegido al destinatario B del que conoces su clave pública (nb, eb), usando bloques del tamaño que debes calcular. Quitas letras de m que no sean del alfabeto, divides m en bloques, los codificas a Znb, los encriptas, los decodificas en mensajes del alfabeto y los unes en un nuevo String que es el resultado.
- **String desencriptar(String m, alfabeto):** ahora recibes un mensaje encriptado con tu pública, y el objeto tiene la privada, luego no necesita más información que el mensaje m y el alfabeto en que está escrito. Quitas del mensaje las letras que no sean del alfabeto, haces bloques, codificas cada uno, desencriptas, decodificas y devuelves el mensaje.

MÉTODOS PARA FIRMAR MENSAJES Y PARA COMPROBAR UNA FIRMA:

La firma de un mensaje m consiste en calcular el hash h del mensaje, h= hash(m) que será un número de Zn y que se llama rúbrica del mensaje. Encriptas h con la privada, y como solamente la tiene el dueño, solo él ha podido realizar la firma.

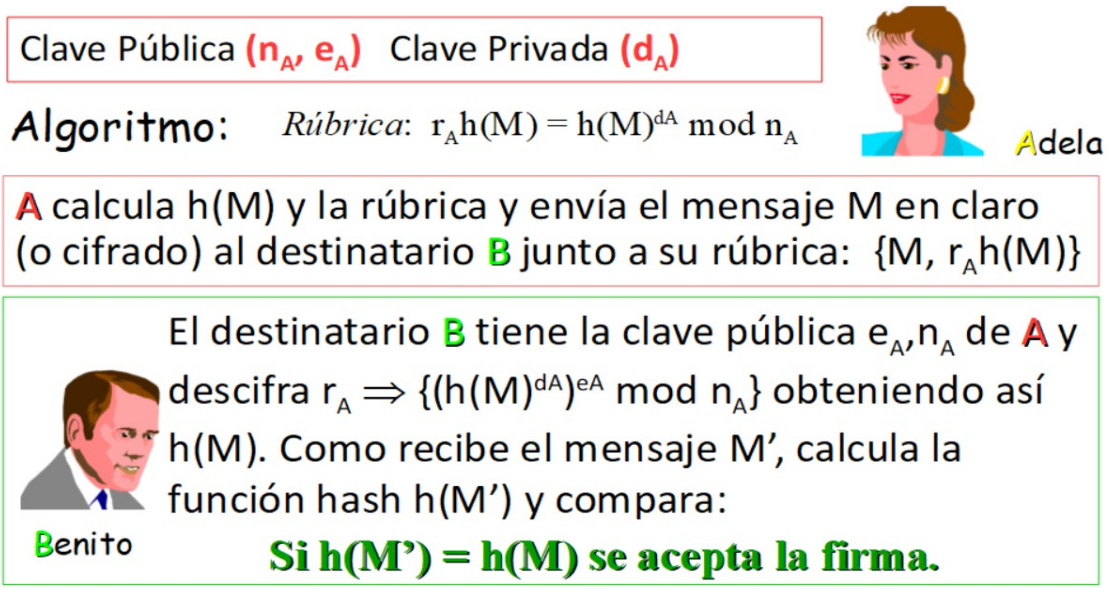
En el ejemplo, B firma un mensaje (que no necesita encriptar, solo firmar), luego calcula su rúbrica y la encripta. Envía a A tanto el mensaje, como la firma:

EJEMPLO DE FIRMA DIGITAL RSA (B → A)



COMPROBACIÓN DE LA FIRMA

FIRMA ELECTRÓNICA DE B → A CON RSA



b) (0.3 PUNTOS) Además de la clase **SimpleRSA**, haz el programa **U04E08** que permita usar la clase, dando la siguiente funcionalidad a la que se accede a través de este menú:

1. Configurar el Alfabeto.
2 Configurar el RSA propio (manual o aleatorio).
3 Configurar RSA de otro (parte pública).
3 Encriptar mensaje para otro.
4 Desencriptar mensaje para mí.
5 Firmar un mensaje.
6 Comprobar firma de un mensaje de otro.
7. Salir.

EJERCICIO 9. MEJORAR EL ALGORITMO moverA(f,c).

Haz una copia al método **moverA(f,c)** del ejercicio del laberinto y la renombras como **moverA1(f,c)**. Vamos a intentar mejorarlo.

a) (0.5 PUNTOS) Una posible mejora es modificar la condición del segundo **IF** y usar un mapa que le proporcione memoria de largo plazo al algoritmo. No usar solo la condición de que ***SI (f,c) está en el camino que estoy explorando ahora mismo ya no lo doy por un lugar bueno***. Tener más memoria: ***si (f,c) es un lugar donde ya he estado alguna vez*** (aunque no sea el camino actual que estoy probando), lo doy por malo y devuelvo false (camino poco prometedor).

Para poder hacerlo, cada vez que el algoritmo llegue a un (f,c) donde se ve obligado a hacer **back**, anota antes en otro laberinto de booleanos, un indicador que se lo recuerde. Puedes fabricar el mapa de booleanos en el propio constructor y almacenarlo en una variable miembro llamada **back** por ejemplo. Cada posición (f,c) del laberinto que sea pared o la salida estará inicialmente a true (hay que hacer back) y cada posición libre y la meta estarán a false.

En el segundo if, si está en una (f,c) donde ya ha hecho back (back[f][c] a true), que salga devolviendo false, para no probar otra vez lo mismo. Con esta mejora, da el mismo resultado pero unas 3000 veces más rápido a costa de un poco de consumo extra de memoria.

Por último, al hacer back, marca la (f,c) a true para recordar que no es buena, por si se llega de nuevo a ella. Entrega código del segundo método y captura de resolver un laberinto igual al resuelto por el algoritmo anterior.

b) (0.5 PUNTOS) ELIMINA RECURSIVIDAD EN moverA2(f,c).

Haz una copia al método **moverA1(f,c)** y elimina completamente la recursividad cambiándola por un bucle del tipo: salir= false; while (!salir) { ... }

No tener recursividad tiene un precio: no sabremos el estado en que estábamos cuando se haga back (perdemos el **tracking**, la ventaja de tener la memoria temporal de las variables locales, eso nos lo proporciona la pila del proceso que en cada llamada crea variables locales para los parámetros f,c y el resto de variables como el movimiento que estábamos probando, la sentencia que debía ejecutarse a continuación...). ¿Cómo superar este inconveniente?

Tendremos que fabricarnos nuestra propia pila, algo que nos permita recordar por donde tenemos que seguir probando. La forma más sencilla es usar la variable que contiene la ruta

con los movimientos realizados. Si es un string, como la vamos a usar de forma intensiva (añadiendo, quitando, etc.) **nos interesa que esos cambios sean rápidos**, así que en vez de usar un objeto inmutable (String) **usa uno mutable** (**StringBuilder**) en su lugar. Una pila siempre añade (**push**) y quita (**pop**) información por el final. Debes transformar el algoritmo original que usaba recursividad:

```
// Intentar ir al norte
intento = moverA(fA-1, cA)
SI intento ENTONCES
    ruta= 'N' + ruta
    RETURN true;
FINSI
   :
```

Por algo similar a esto:

```
ruta = "";
movimiento = 'E';  // Empezar por el este ya que la meta estará en esa dirección
posicion = (fS,0); // Posición inicial de la que salir (Habrá que validar donde vamos)
MIENTRAS NO salir HAZ
        SI movimiento='E'
                SI (fA, cA+1) Es meta o correcta
                ENTONCES
                        ruta = ruta + 'E';
                        posicion = (fA, cA+1)
                        SI posicion es meta ENTONCES DEVOLVER true; FINSI;
                        // Dibujar camino...
                SINO movimiento= 'S';
                FINSI
        FINSI;
        SI movimiento = 'S'
                SI (fA+1,cA) Es meta o correcta
                ENTONCES
                        ruta = ruta + 'S';  // Recordar el movimiento: push(movimiento)
                        posicion = (fA+1, cA)
                        SI (fA+1,cA) es meta ENTONCES DEVOLVER true; FINSI;
                        // Dibujar camino...
                        movimiento= 'E'; // Para la siguiente posición
                SINO movimiento = 'N';
        FINSI
        SI ...  // movimiento Norte
         :
        SI ...  // movimiento Oeste
           SI (fA,cA-1) Es la meta o correcta
           ENTONCES
                ...
           SINO  // lugar sin futuro, probados todos -> back
                REPETIR
                        // Saca el último movimiento de la ruta
                        SI ruta no vacía
                                ultimo = pop(ruta)
                                ruta = ruta menos el último movimiento
                                // Quitar (fA,cA) como camino y marcarlo como malo en el mapa
                                // Deshaz el movimiento que nos ha llevado hasta aquí
                                SI ultimo = 'E'
                                ENTONCES posicion = (fA, cA-1)
                                SINO SI ultimo='S'
                                ENTONCES posicion = (fA-1, cA)
                                SINO SI ultimo='N'
                                ENTONCES posicion = (fA+1, cA)
                                SINO  // Ultimo es 'O' y ya no hay más que probar
                                        posicion =(fA, cA+1)
                                FINSI
                        FINSI
```

```
                HASTA QUE ultimo distinto de 'O' o ruta vacía;
                // Coger el siguiente movimiento a probar en la posición anterior
                SI ruta vacía ENTONCES DEVOLVER false; FINSI
                SINO
                        SI ultimo = 'E' ENTONCES movimiento = 'S'
                        SINO SI ultimo='S' ENTONCES movimiento = 'N'
                        SINO movimiento = 'O'   // No puede ser O pq es el último
                        FINSI
                FINSI
        FINSI
FINMIENTRAS
```

La idea básica de esta solución es usar el string **ruta** como una pila, añadiendo y quitando (push() y pop()) movimientos solo por el final, quitamos en back y deshacemos a donde nos llevó ese movimiento continuando por el siguiente si lo hay o volviendo a hacer back si no hay más por probar (también lo puedes marcar como no visitable en el mapa de booleanos). Ten en cuenta además que no he marcado el camino en el laberinto, debes hacerlo tu mismo.

Pasa el código de **moverA2()** y una captura de pantalla donde aparezca el tiempo que le cuesta intentar solucionar el mismo laberinto que **moverA()** y **moverA1()** para comparar su comportamiento. Si ves la figura, verás que es el más rápido de los que llevamos.

EJERCICIO 10. UN JUEGO EN EL TEMINAL.

Crea una clase llamada Conjunto que represente a un conjunto de números enteros. La clase **no tendrá límite a la cantidad de enteros que puede almacenar y no consumirá más memoria de la necesaria** y debe respetar la idea básica de los conjuntos que consiste en que **no tiene elementos repetidos**. Tu código debe conseguir estos dos objetivos además de implementar estas operaciones:

- 3 Constructores:
  - Uno que reciba un array de enteros.
  - Otro el constructor por defecto, que crea un conjunto vacío (sin elementos).
  - Otro que reciba de argumento un número arbitrario de enteros.
- Operaciones externas:
  - boolean esVacio(): devuelve true si no tiene elementos.
  - boolean pertenece(int elemento): devuelve true si elemento está en el conjunto.
  - boolean contiene(Conjunto c): devuelve true si todos los elementos de c están dentro del conjunto que recibe la llamada.
  - void inserta(int elemento): añade elemento al conjunto (si no está ya).
  - void borra(int elemento): elimina elemento del conjunto (si lo tiene).
  - int cardinal(): devuelve el número de elementos que tiene el conjunto.
  - Conjunto union(Conjunto c): devuelve un nuevo conjunto que tiene sus elementos y los de elementos de c (pero sin repetidos, recuerda).
  - Conjunto interseccion(Conjunto c): devuelve un nuevo conjunto con los elementos comunes con c (los tiene él y también los tiene c).
  - Conjunto diferencia(Conjunto c): devuelve un nuevo conjunto con los elementos del conjunto menos los que estén en c, que se eliminan.
- Sobreescribe equals() y toString().
- Añade un main y realiza:

```
Conjunto c1 = new Conjunto();
Conjunto c2 = new Conjunto(1, 2, 3, 4, 5, 6, 6);
Conjunto c3 = new Conjunto(2, 4, 6, 8);
Conjunto u, i, d;
u = c2.union(c3);
i = c2.interseccion(c3);
d = c2.diferencia(c3);
d.inserta(7);
Imprime c1, c2, c3, u, i y d
Imprime d.esVacio() y u. cardinal
```

EJERCICIOS OPCIONALES

OPCIONAL (+4 PUNTOS). UN JUEGO EN EL TEMINAL.

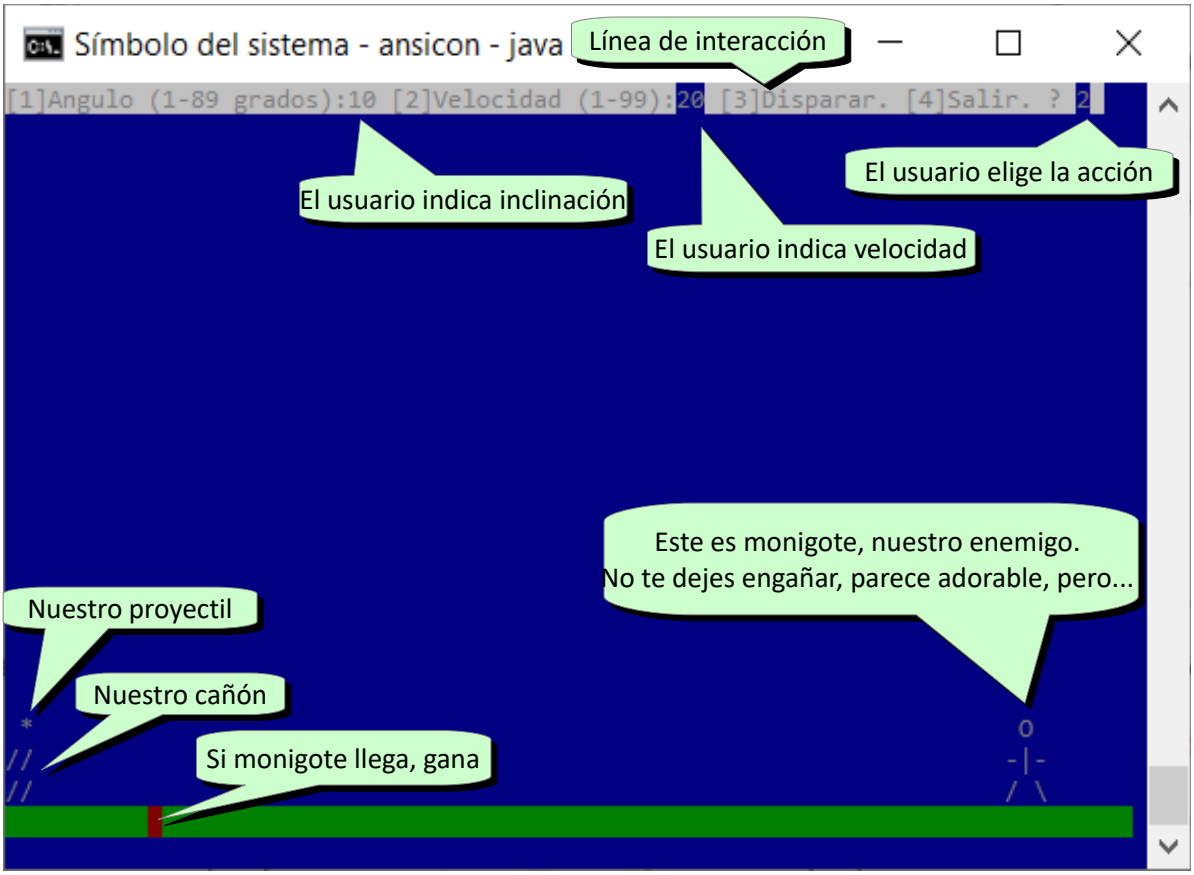
Usando la posibilidad de escribir en la parte del terminal que quieras que conseguiste con la clase **MiCo**, un poco de física que ya has utilizado en algunos problemas anteriores y la imaginación, podemos crear un juego que consista en intentar matar a un personaje que se puede mover tras cada disparo de un cañón, antes de que se acerque demasiado a nosotros. Ganamos si el proyectil alcanza al monigote. Perdemos, si el monigote consigue traspasar cierto límite acercándose a nuestro cañón. El algoritmo del juego en pseudocódigo será:

```
hayDisparo <- false  // Controla si se dibuja un disparo
opción <- 0;         // Lo que ha elegido el jugador
dibujar el escenario(hayDisparo);
MIENTRAS no se de la orden de salir (opción distinta de 4) HACER
    PASO 1: Esperar opción elegida por el usuario.
    PASO 2: Aplicar la opción.
        - SI opcion es cambiar el ángulo del cañón: leer y validar el nuevo ángulo.
        - SI la orden es cambiar la velocidad: leer y validar la nueva velocidad.
        - SI la orden es disparar:
            - Calcular los datos del disparo.
            - hayDisparo <- true;
    PASO 3: Cambiar posición de monigote.
    PASO 4: Dibujar escenario(hayDisparo):
        - Dibujar la zona de interacción con usuario.
        - Dibujar escenario, cañón y bala.
        - Dibujar monigote en su posición.
        - SI hayDisparo ENTONCES dibujarlo y comprobar si da a monigote.
FinMientras;
```

EL ESCENARIO DE LA BATALLA.

Definimos un campo de batalla de 25 líneas de altura (filas, eje vertical) y 80 columnas de ancho (eje horizontal X). Cualquier posición en el campo de batalla se indica con las

coordenadas (fila, columna). La primera coordenada (0,0) está situada en la esquina superior izquierda y a medida que bajamos aumentan las filas (1,0), (2,0) ... hasta (24,0).



Dibujar el escenario: Pon color de texto negro y fondo azul usando **MiCo**. Borramos la pantalla. Ahora vamos a hacer una base que sirva de suelo, para ello ponemos el cursor en la coordenada (0, 24) y cambiamos el color de fondo a verde para dibujar un suelo escribiendo 10 espacios en blanco (sin saltar). Cambiamos el color de fondo a rojo, escribimos un espacio para marcar la meta a la que debe llegar monigote para gabnar, y volvemos a poner el color de fondo a verde para completar el suelo con **ancho-11** espacios (el último hueco no, para no desplazar).

Ahora nos posicionamos en la primera fila (0,0) y cambiamos color de texto a negro y color de fondo a gris. Escribimos con formato el menú para interactuar con el jugador.

El monigote (nuestro enemigo):

```
 O
-|-
/ \
```

Lo dibujamos con letras. Su posición la asociamos con las coordenadas donde está su cabeza en las variables estáticas **moniF** (fila del monigote) y **moniC** (columna del monigote). Y lo digujamos a partir de esos datos: en `(moniC, moniF)` escribo una "O", en `(moniC+1, moniF-1)` escribo "-|-" y en `(moniC-2, moniF+1)` escribo "/ \".

Para borrarlo: escribo espacios en blanco en esas posiciones

Si te haces métodos que lo dibujen y borren pasando las coordenadas donde tiene la cabeza, simplificarás el código. Su posición inicial será en (72, 22).

Movimientos: generamos un un nº aleatorio entre [0, 1). Como los nºs tienen la misma probabilidad de caer en cualquier lugar de este intervalo (siguen una distribución uniforme) podemos hacer acciones que sucedan de forma impredecible, pero siguiendo cierta tendencia:

A izq.2 (30%) A izq.1(30%) A der.1(30%) A der.2(10%)

Si el nº es mayor o igual de 0.9 se mueve a la derecha 2 posiciones (el 10% de las veces), si no, si es mayor o igual de 0.6 (el 30% de las veces) se mueve a la derecha una posición. Si no, si es mayor o igual de 0,3 (el 30% de las veces) se mueve a la izquierda dos posiciones y en otro caso (el 30% de las veces) avanza hacia la izquierda una posición.

Con esta estrategia, su tendencia será acercarse al cañón (probabilidad del 60%) frente a alejarse (40%).

Aunque tenga tendencia a moverse hacia la izquierda, al ser algo aleatorio, podría intentar salir de la pantalla, si sobrepasa la columna ANCHO - 3 (si moniC >= ANCHO-3). En ese caso, lo bloqueamos (moniC = ANCHO - 3). Si al moverse a la izquierda llega a una columna igual o inferior a 10 (META), hemos perdido, monigote nos ha derrotado!! :-(

La bala de cañón:
Será un asterisco. Su posición inicial antes de disparar será la (2, 21). Su movimiento en el disparo lo determinarán las leyes de la física, según la velocidad inicial v0 y ángulo (alfa) con que se dispara.

El cañón: Se dibuja con letras y tomaremos como referencia la esquina superior izquierda. Como no se mueve como monigote, y literalmente son cuatro letras, no merece la pena hacer un método para dibujarlo. Su posición inicial es la (1, 22). El dibujo es sencillo, dibujar la cadena "//" en su posición y en la (0, 23) y escribir el asterisco (la bala) en la (2, 21).

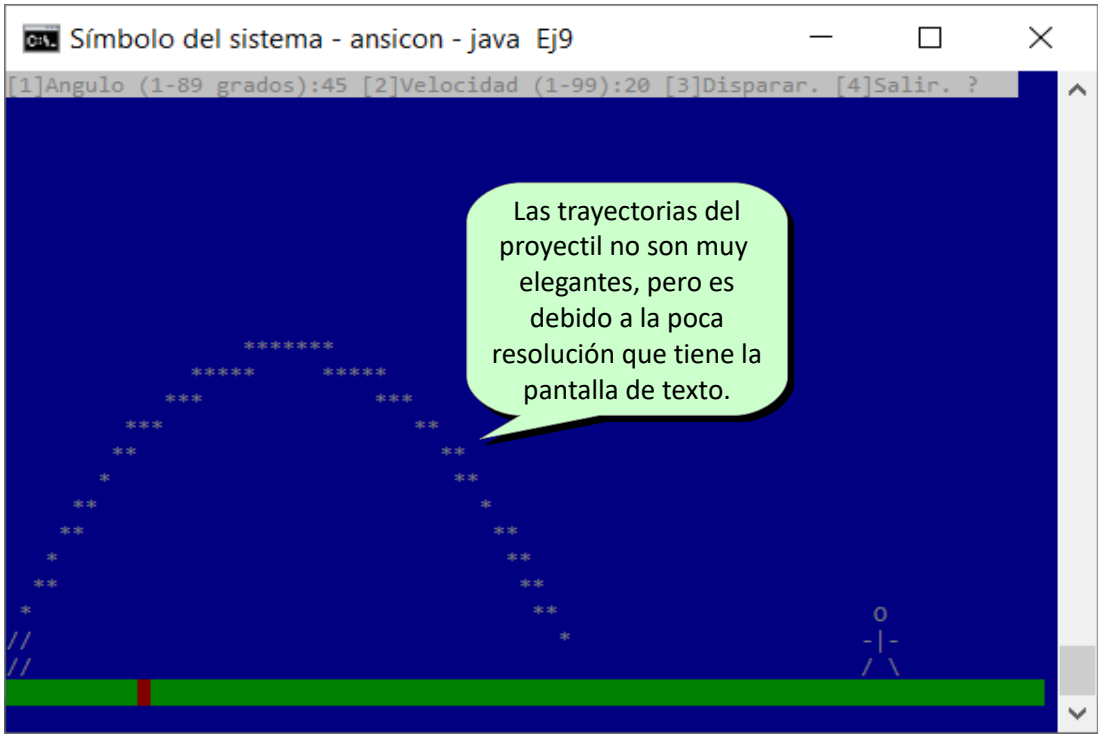
La entrada de datos:
En la primera fila, coordenada (0,0) mostramos un menú:

```
"[1]Angulo (1-89 grados): 20  [2]Velocidad(5-10): 30 [3]Disparar. [4]Salir. ? 3 "
```

- Tras la interrogación leeremos lo que quiere hacer el usuario.
- Si teclea el entero 1, lo llevamos a la posición donde se lee el ángulo (donde aparece en azul un 20) y leeremos un entero con la velocidad validando que esté entre 1 y 89. No le dejamos 0 porque le hace un agujero en el pecho a monigote, y queremos matarlo pero que no sufra (que buena gente somos...). Tampoco dejamos que sea 90 porque la bala subirá y bajará de forma vertical y tampoco queremos suicidarnos.
- Si teclea el nº 2, lo llevamos a donde se indica la velocidad y validamos entre 1 y 99.
- Si pulsa 4, el juego acaba, ponemos la opción a 4 para salir del bucle del juego y fuera del bucle ponemos color de fondo negro y color de texto blanco y borramos la pantalla.

- Si escoge la opción 3, ahí viene lo interesante. Usa las fórmulas de física del tiro parabólico que viste en el ejercicio de la práctica de la unidad 2. Simplemente ponemos un indicador de que hay que disparar y ahora comentamos como calcular los datos para dibujar el disparo.

Sabiendo la velocidad inicial y el ángulo que forma el cañón con respecto al suelo (alfa), podemos calcular el resto de cosas usando las fórmulas de Newton que ya usaste en un ejercicio de la unidad 2. Si observas, nada más escoger la opción de disparar, ya podemos saber si va a destrozar a monigote o no le va a dar. Si coordenadas de monigote, coinciden con las coordenadas que ocupa la bala pasado el tiempo de vuelo, monigote ha sido fulminado. Para darle a monigote una ventaja, y hacer el juego un poco más atractivo, vamos a darle la posibilidad de que cambie su posición antes de realizar el disparo. Así que debemos comprobar si le damos, cuando dibujemos el disparo, no antes, dando a monigote la posibilidad de cambiar su posición.



El tiempo de vuelo es pequeño y si escribimos y borramos la bala para que se mueva, no veremos la bala volar de un sitio a otro, lo hace demasiado rápido. Para que el jugador se haga una idea de donde caen sus disparos, usaremos balas trazadoras, no borramos su posición anterior. Simularemos el vuelo de la bala desde su posición hasta que caiga a la altura de la cabeza de monigote, dibujando las posiciones que va recorriendo la bala en su trayectoria:

- Dividimos el tiempo de vuelo en 100 trozos, definiendo intervalos de incT = tv/100.
- Calculamos donde está la bala cada uno de esos 100 momentos t0, t0+incT, t0+2incT, ... t0+ 99*incT, donde se queda la bala. Calcularemos sus coordenadas en ese tiempo en el mundo real. Las pasaremos a coordenadas de pantalla y dibujaremos el asterisco sin borrar.
- Si las coordenadas de pantalla de su posición están fuera del escenario, las dejamos dentro del límite del escenario (tanto filas como columnas).

Aquí tienes un disparo certero que acaba con monigote:

OPCIONAL (+5 PUNTOS). ALGORITMOS GENÉTICOS.

Los Pájaros y las Ovejas. Fuente "AI Techniques For Games", Premier Press.

Todos los organismos vivos son esencialmente una gran colección de células. Cada célula contiene el mismo sistema de secuencias de **ADN**, llamadas **cromosomas**. El ADN que contiene un cromosoma tiene dos cadenas de elementos básicos enroscadas en forma de doble hélice, y los filamentos o cadenas están conectadas la una a la otra formando una trenza que se retuerce en espiral, formando la familiar hélice del ADN mostrada en la figura 1.

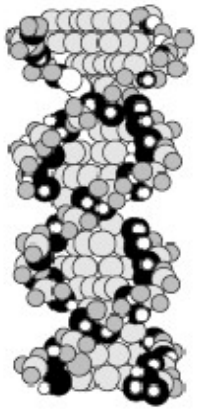


figura 1: La doble hélice de la vida.

Los cromosomas individuales están formados por bloques más pequeños, llamados **genes**, que a su vez se forman con sustancias llamadas **nucleótidos**. Hay solamente 4 nucleótidos: **tinina**, **adenina**, **citosina** y **guanina**. Se abrevian como **T**, **A**, **C** y **G**. Estos nucleótidos se unen juntos en largas cadenas de genes, y cada gen codifica cierto rasgo de un organismo, tal como el color del pelo o la forma del oído. Las diversas configuraciones que un gen puede tener-para el ejemplo del color del pelo, serían marrón, rubio o moreno- se llaman **alelos**, y su posición física a lo largo de la longitud del

Nota Histórica

Los algoritmos genéticos los crea Jhon Holland, que tuvo la idea a principios de los años 1960. No intentó hacerlos funcionar en un ordenador, los probaba con lápiz y papel. Hasta que un estudiante suyo escribió un programa en un ordenador personal y el mundo finalmente vio sus posibilidades.

cromosoma se llama **locus**.

La colección de cromosomas dentro de una célula contiene toda la información necesaria para reproducir ese organismo. Puedes copiar un organismo, como una oveja, con la información contenida en apenas una célula de sangre de la oveja y crear una nueva oveja. Las nuevas ovejas serán idénticas en todos los conceptos a la oveja donante de la célula. Esta colección de cromosomas se conoce como el **genoma del organismo**. El estado de los alelos en un genoma particular se conoce como **genotipo** de ese organismo. Éstas son las instrucciones codificadas físicamente que dan lugar al organismo real, que se llama el **fenotipo**. Tú y yo somos fenotipos. Nuestro ADN lleva nuestro genotipo. Explicado de otra manera, el modelo y las configuraciones de un diseño de coche es un genotipo, mientras que el coche acabado que sale de la cadena de producción es un fenotipo. O de otra forma: una clase es el genotipo y cada uno de sus objetos es un fenotipo. Al diseño se le puede llamar un genoma.

Vamos con la evolución. Si eres el tipo de persona que mira el mundo exterior de vez en cuando, alejado de la pantalla del ordenador (se que existe ese mundo porque mis amigos me lo han dicho), habrás notado que ese mundo es la casa de un gran número de criaturas y millones de plantas de todos los tamaños, formas y colores. Desde los organismos unicelulares microscópicos a grandes formas de vida. **Un organismo se considera acertado** si se adapta a sobrevivir y genera nuevos organismos, que, supuestamente, podrán volver a reproducirse. Para conseguirlo, el organismo debe ser bueno en muchas tareas, tales como encontrar alimentos, defenderse de sus depredadores y hacerse atractivo a sus compañeros potenciales. Todas estas cualidades dependen en cierta manera del genotipo, del modelo de criatura. Algunos de los genes del organismo darán lugar a cualidades que le ayudarán a ser un organismo adaptado, y algunos de sus genes pueden obstaculizarle en vez de ayudarle. **A la medida del éxito de un organismo se le llama su aptitud (no confundir con actitud).** Cuanto más apto es el organismo, más probable es que produzca descendientes. Ahora viene la parte romántica...

Cuando dos organismos se acoplan y se reproducen, sus cromosomas se mezclan juntos para crear un nuevo sistema de cromosomas completo, que consiste en genes de ambos padres. Este proceso se llama **recombinación** o **cruce**. Esto podría significar que el descendiente hereda principalmente los genes buenos, en este caso puede desenvolverse mejor que sus padres (por ejemplo, tiene mejores mecanismos de defensa contra los depredadores), o puede heredar principalmente los genes malos, en este caso, puede incluso no llegar a reproducirse.

Lo importante es que cuanto más adaptado esté el descendiente, más probable es que se reproduzca y pase sus propios genes a sus descendientes. Cada generación, por lo tanto, mostrará una tendencia a ser mejor en la supervivencia y la adaptación.

Como ejemplo rápido y muy simple, imagina a criaturas femeninas a las que les atraen solamente criaturas masculinas con los ojos grandes. Básicamente, cuanto más grandes son los ojos, mayor es la probabilidad de que una criatura masculina tenga éxito en "cortejar" a las hembras. Podría decirse que la aptitud de una criatura es proporcional

al diámetro de sus ojos. Así pues, si comienzas con una población de estas criaturas masculinas, todas con diferentes tamaños de ojos, puedes ver que el gen en las criaturas masculinas se fija para construir un ojo más grande cuando el animal está evolucionando, porque así es más probable ser copiado a la generación siguiente que cuando el gen o alelo está configurado para desarrollar un ojo pequeño.

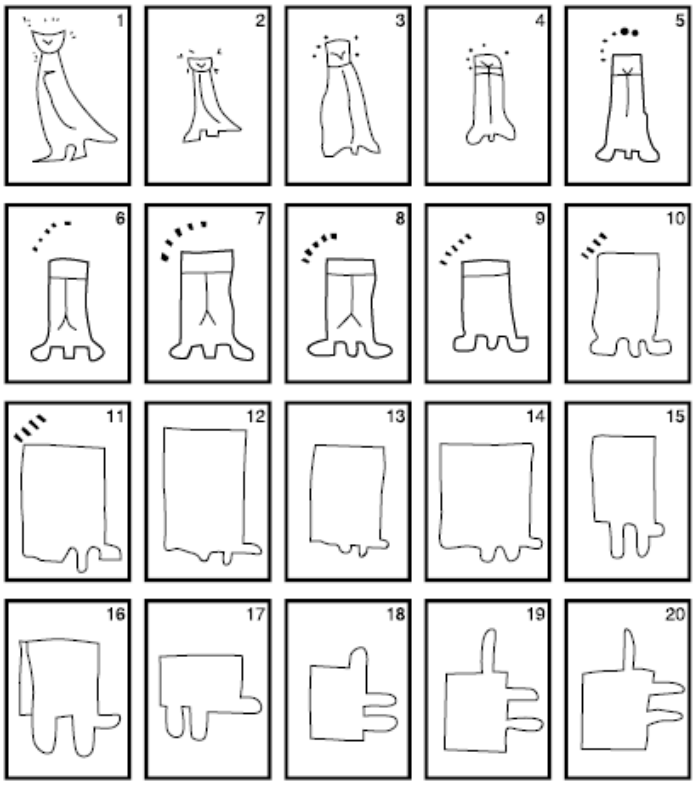


figura 2: Un experimento de transferencia de información. (Illusions de Thames and Hudson.)

Sigue esta tendencia después de muchas generaciones, y los ojos más grandes van a ser dominantes en la población masculina. Puedes decir que, tras cierto plazo, las criaturas convergen hacia un genotipo particular.

Sin embargo, si ésto es lo único que influye durante la reproducción, incluso después de muchos millares de generaciones, el tamaño del ojo del miembro más apto solamente podrá ser tan grande como el ojo más grande de la población inicial. Pero sólo hay que observar la naturaleza, para ver que las tendencias como el tamaño del ojo cambian realmente de generación en generación.

Esto sucede porque cuando los genes se pasan al descendiente, hay una **probabilidad muy pequeña de que ocurra un error y los genes cambien**. Es como el juego de los susurros, en que un mensaje se va pasando susurrado en una línea de personas. La primera persona susurra una frase en el oído de la segunda y así sucesivamente, hasta que la última persona en la línea dice el mensaje que oyó. Generalmente, el mensaje final no tendrá nada que ver con el original. Estos tipos de errores ocurren en cualquier clase de información pasada a partir de un sistema al siguiente. Un ejemplo asombroso de esto son los dibujos mostrados en la figura 2. Éstos son los resultados de una prueba en la cual un dibujo de un pájaro (arriba izquierda) se pasa a una persona para que lo copie y entonces esa copia se pasa a la persona siguiente, hasta que se alcanza la notable transformación del último dibujo. Si estás en una reunión de quince amigos o así, y haces esto, parece increíble que el dibujo inicial pueda cambiar tanto.

Podrías decir que la frase o el dibujo se transforman de persona a persona, y el mismo tipo de mutación puede ocurrirle a los genes durante el proceso de reproducción. **La probabilidad de que ocurra una mutación es pequeña, sin embargo, significativa sobre un número**

Hecho interesante

Incluso las monedas antiguas eran propensas a este tipo de pérdida de información. Las primeras monedas célticas y Teutónicas fueron falsificadas profusamente, y una moneda original que tenía la cara del emperador de turno en ella, podrías encontrarla al cabo del tiempo en otra ciudad con la forma de un caballo o de un frutero. ¡No se necesitaban dispositivos ultravioleta de alta tecnología para detectar una falsificación!

bastante grande de generaciones.

Algunas mutaciones serán perjudiciales (probablemente la mayoría), algunas no harán ningún cambio en la aptitud de la criatura, pero unas pocas darán a la criatura una ventaja sobre el resto de criaturas de su tipo. En el ejemplo anterior, cualquier mutación que ocurra en los genes de una criatura, que de lugar a un ojo de un diámetro más grande, va a hacer que lo perciban como un supermodelo de Vogue en comparación con el resto de la población. Por lo tanto, la tendencia será que se perpetúen los genes responsables de los ojos más y más grandes. ¡Después de millares de generaciones, este proceso de la evolución puede producir ojos tan grandes como platos! Figura 3.

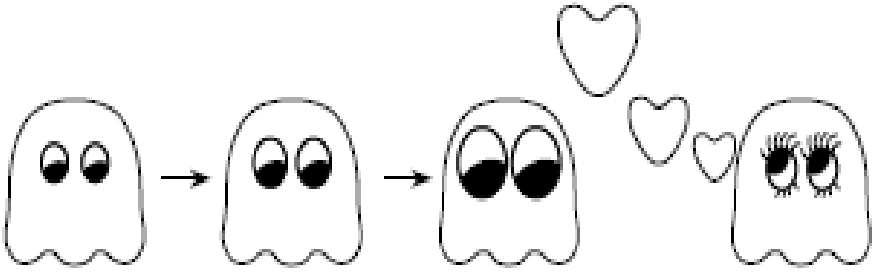


Figura 3: La evolución de un Adonis.

Además de poder alterar una característica existente, los mecanismos de la evolución pueden dar a luz características totalmente nuevas. Tomemos la evolución del ojo como ejemplo… Imagina una época en que las criaturas no tenían ojos. En esos días, estas criaturas se movían por su ambiente y evitaban a sus depredadores por el olor y el tacto. Y eran bastante buenas haciéndolo, porque habían estado haciéndolo así durante millares de generaciones. En aquella época, los varones con narices y manos grandes eran populares entre las muchachas de su especie.

Entonces un día, cuando dos de las criaturas se acoplaron, ocurrió una mutación en los genes que proporciona un primer modelo para las células de la piel. Esta mutación dio lugar al descendiente que desarrolló una célula de la piel en su cabeza, que era levemente sensible a la luz, apenas lo bastante como para que el descendiente pudiera saber si en su ambiente había luz u oscuridad. Esto le daba una leve ventaja, porque si un depredador bloqueba la luz, la criatura sabría qué hacer para protegerse. La célula de la piel indicaba si era de noche o de día, o si estaba en el suelo o bajo tierra, puede dar una ventaja en la caza y la alimentación. Este nuevo tipo de célula de la piel dio a las criaturas una leve ventaja sobre el resto de la población y, por lo tanto, una mejor probabilidad de sobrevivir y de reproducirse. En cierto plazo, debido a los mecanismos de la evolución, más criaturas tendrán cromosomas que incluyen el gen de la célula de la piel sensible a la luz.

Ahora, si extrapolas esto un poco e imaginas otras mutaciones ventajosas en el mismo gen de la célula de la piel, puedes ver como al pasar muchas generaciones, este área de ligera sensibilidad puede crecer más y obtener otras características, como una lente, células adicionales para detectar colores, etc. Imagina una mutación que de a la criatura no sólo una de estas áreas de sensibilidad lumínica, sino dos, y por lo tanto, regala a la criatura la visión en estéreo. La visión en estéreo es un paso adelante enorme para un organismo, porque ahora

puede decir exactamente cómo de lejos están los objetos (puede calcular distancias).

Por supuesto, algunos individuos también tendrán mutaciones de esos mismos genes, que dan lugar a características perjudiciales, pero lo importante es que esas criaturas no estarán tan adaptadas como sus primos con sus nuevos y mejorados ojos, y por lo tanto, eventualmente morirán con más probabilidad, haciendo que sus genes poco adaptados desaparezcan. Solamente los genes más adaptados perduran. Puedes pensar en cualquier características que sea ventajosas para sus dueños en la naturaleza e imaginar cómo se ha podido desarrollar a través de una miríada de minúsculas mutaciones. ¿Increíble, no? Así pues, estos mecanismos de recombinación y mutación ilustran cómo trabaja la evolución, y espero que ahora entiendas cómo los organismos pueden desarrollar diversos tipos de características para ayudarles a estar más adaptados dentro de su ambiente.

Copiar La Evolución En Tu Computadora.

El funcionamiento de los algoritmos genéticos es esencialmente mimético al de la evolución. Primero, hay que pensar una manera de codificar cualquier solución potencial de un problema como un cromosoma "digital" (bueno, los de la naturaleza tienen cadenas de "ACTG" y los del ordenador cadenas de "01" y aunque una alcachofa y un ser humano son aparentemente muy diferentes, sus genes son esencialmente lo mismo. Bueno pues aunque la solución a una ecuación hiperbólica o la solución de encontrar la salida en un laberinto parecen bastante diferentes, esencialmente son lo mismo: ceros y unos.

Cuando ya sabes como representar las soluciones de un problema (los cromosomas), creas una población de cromosomas al azar (en principio cada uno representa una solución candidata diferente) y aplicas la evolución durante un cierto tiempo, "criando" a individuos más aptos y aplicando una pequeña mutación aquí y allí. Con un poco de suerte, tras muchas generaciones, el algoritmo genético converge hacia las mejores soluciones.

Los algoritmos genéticos no garantizan una solución, ni garantizan encontrar la mejor solución, pero si se utilizan de manera correcta, generalmente funcionan bien. **Lo mejor de los algoritmos genéticos es que no necesitas saber solucionar un problema,** necesitas solamente saber plantearlo de manera que se pueda utilizar el mecanismo de la evolución.

Típicamente, los cromosomas se codifican como serie de bits binarios. Al principio de una ejecución, creas una población de cromosomas (soluciones de un problema) que tienen sus bits generados al azar. La longitud del cromosoma es generalmente fija para toda la población. Por ejemplo, un cromosoma digital de longitud 20 puede parecerse a esto: 01010010100101001111.

Lo importante es que cada cromosoma está codificado de tal manera que la cadena de bits se puede descifrar para representar una solución a un problema concreto. Puede ser una solución muy pobre, o puede ser una solución perfecta, pero cada cromosoma representa una solución posible (en la codificación de cada problema).

Generalmente generar la población inicial es terrible. De todas formas, puedes creala al azar (digamos unas 100 soluciones aleatorias) y entonces haces esto:

Repetir hasta que se encuentre una solución aceptable:

1. Mira cada cromosoma (solución) para ver cómo de bueno es solucionando el problema y le asignas una cantidad o medida ente [0,1] a su aptitud (a lo bien adaptado que está), 0 muy mal adaptado y 1 perfectamente adaptado.
2. Selecciona a dos miembros de la población actual. La probabilidad de ser seleccionado será proporcional a la aptitud (cuanto más alta sea su aptitud, mayor será la probabilidad de ser seleccionado). Un método común para hacer esto es la llamada ***selección de la rueda de ruleta***.
3. Dependiendo de la **tasa de cruce**, cruzar los bits de cada pareja de cromosomas elegidos con un proceso aleatorio.
4. Cambiar cada bit del cromosoma elegido, según la tasa de mutación.
5. Repetir los pasos 2, 3 y 4 hasta que una nueva población de cientos de miembros se haya creado.

Fin de Repetir.

A cada ejecución de este bucle repetir (del algoritmo) se le llama **una generación** (pasos del 1 al 5). A la ejecución del algoritmo 1000 veces (mil generaciones), se le llama **una época**.

¿Qué es la Selección de la Rueda de Ruleta?

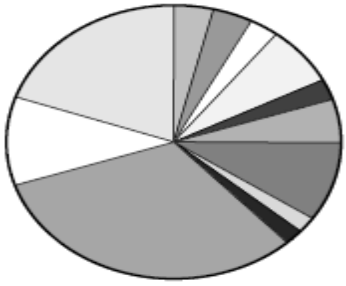


Figura 4: Rueda de ruleta para selección de cromosomas.

Es un método para elegir a miembros de la población de manera que la probabilidad de ser elegidos para procrear (pasar sus genes) sea proporcional a la aptitud de sus cromosomas, que de alguna forma mide su adaptación, es decir, lo buena solución que es para un problema.

El método no garantiza que el miembro más apto vaya a pasar a la generación siguiente (es aleatorio), simplemente que ese miembro tiene una probabilidad muy buena de hacerlo.

Funciona así: la suma total de aptitudes de la población representa una rueda de ruleta (figura 4). Ahora, asignas un sector de la rueda a cada miembro de la población. El tamaño del sector (el ángulo) es proporcional a su aptitud, si un miembro tiene una aptitud mayor, tendrá un sector mayor. Ahora, para elegir un cromosoma, lo que tienes que hacer es hacer girar la ruleta, tirar una bola y coger el cromosoma en cuyo sector ha caido.

¿Qué es la Tasa de Cruce?

La tasa de cruce es simplemente la probabilidad de que dos cromosomas elegidos intercambien sus bits para producir el nuevo descendiente a partir de los dos. La experimentación ha demostrado que **un buen valor está alrededor de 0.7**, otros problemas dan mejores resultados con valores más altos o más bajos.

Cada vez que eliges dos cromosomas de una población, hay que decidir si procrean generando un número al azar entre 0 y 1. Si el número es más bajo que la tasa de cruce (0.7), eliges una

posición al azar a lo largo de la longitud del cromosoma e intercambias todos los bits tras este punto. Por ejemplo, dados los dos cromosomas siguientes:

```
mami -> 10001001110010010
papi -> 01010001001000011
```

Se elige una posición al azar, digamos que sale 10, e intercambias todos los bits a la derecha de ese punto. Los cromosomas se convierten (he dejado un espacio en el punto del cruce). De forma que de 2 bichos, sacas otros dos con sus genes cruzados:

```
hijo 1 →  100010011 01000011
hijo 2 →  010100010 10010010
```

¿Qué es la Tasa de Mutación?

La tasa de mutación es la probabilidad de que un trozo de cromosoma sufra un cambio (un 0 se convierte en 1 y un 1 se convierte en 0). Este es **un valor generalmente muy bajo para los genes codificados en binario, por ejemplo 0.001.** Así pues, siempre que elijas los cromosomas de la población, primero haces el cruce, y entonces, recorres los bits de los cromosomas y compruebas si es necesario transformar los descendientes. Si tu algoritmo trabaja con bytes en vez de con bits, multiplica por 8 la tasa de mutación, 0.008 por ejemplo.

UNIR TODOS LOS ELEMENTOS PARA BUSCAR LA META DEL LABERINTO.

Aprovecha el código que ya tienes para generar laberintos. Lo único que vas a cambiar es la forma de encontrar el camino que nos lleve a la salida. Crea un nuevo método **boolean moverAG()** que prepare y coordine lo necesario en un laberinto para encontrar la salida, o crea una nueva clase AG desde donde hacerlo. Lo organizas como quieras. Necesitas:

1 PUNTO: un nuevo método al que pasar de parámetro una ruta y compruebe hasta donde se puede llegar al aplicarla en el laberinto. Lo vamos a llamar **double probarRuta(String ruta, Laberinto lab)**. El String ruta contiene los movimientos desde la entrada y podrá tener los caracteres "NESO" que indican movimientos al norte, este, sur y oeste. Calcula la posición donde esa ruta te deja en el laberinto desde (fE,0). Luego devuelve un valor que mida lo lejos que estás todavía de la salida. Cuanto más cerca de la meta consigas estar, más alta es la aptitud. Si alcanzas la meta, recibe la máxima aptitud de 1.

Codificando Cromosomas.

Cada cromosoma representa el movimiento en el laberinto, que se restringe a cuatro direcciones: norte, sur, este y oeste, así que los cromosomas de este problema (las soluciones a este problema) son las secuencias de movimientos en estas cuatro direcciones.

Ahora, todo lo que hay que hacer es colocarnos en la entrada y seguir esas instrucciones. Si una dirección hace que se choque en una pared, a esa instrucción simplemente no se le hace caso y el programa se mueve a la instrucción siguiente. Este proceso continúa hasta que el string de movimientos se agota o alcanza la salida en algún momento. En una colección de centenares de estos cromosomas generados al azar, algunos de ellos podrán alcanzar la salida (una solución), pero la mayor parte fallan.

El algoritmo genético toma una población inicial de estas rutas (unos 100, los cromosomas),

prueba cada uno para ver cómo de cerca se queda de la salida (mides su aptitud o adaptación), y entonces se reproducen los mejores con la esperanza de crear descendientes que permitan mejorar un poco más la especie (los cromosomas, las rutas, las soluciones). Este proceso continúa hasta que se encuentra una solución o hasta que nos quedemos desesperados en una esquina (que puede que suceda).

1 PUNTO: Hay que definir la clase **Genoma** con una estructura que almacene el cromosoma (un string o similar) y un double con su aptitud asociada. Le haces dos constructores: un constructor vacío (sin parámetros) que deja el cromosoma vacío y la aptitud a 0 y otro constructor que recibe un entero, que indica la longitud de la ruta, y debe crear la cadena de acciones de forma aleatoria y con esa longitud. Además getters y setters y el método **double mideAdaptacion(Laberinto la)** que calcule la adaptación al Laberinto la del gen (ruta).

Ahora necesitas hacer una nueva clase/Método llamada **AlgoritmoGenetico (AG) que tenga:**

- Un array de genomas llamado **población**.
- Cuantos individuos va a tener la población.
- Una variable con la tasa de cruce.
- Una variable con la tasa de mutación.
- Una variable con la longitud de cada gen.
- Una variable con la media de la aptitud de los cromosomas de toda la población.
- Un contador de generaciones.
- Un array con la mejor ruta de cada generación.

Necesitas un método **Genoma[] creaPoblacion(int n)** que devuelva un array de n genomas creados de forma aleatoria y los almacenas en poblacion[].

Necesitas un método **Genoma[] reproduce(Genoma mami, Genoma papi)** que devuelva dos genomas bebés en un array resultado de reproducir a mami y papi. Podrían devolver null si no procrean (es aleatorio y depende de la tasa de cruce).

Necesitas un método **boolean mutacion(Genoma g)** que devuelva true si ha cambiado elementos de g o false si no.

Necesitas un método **boolean epoca()** que devuelva true si ha encontrado una solución o false si no.

Épocas.

La chicha del algoritmo genético es el método **epoca()**. Éste es el bucle del algoritmo que se describió antes y es el método fundamental. Debe:

- Actualizar los marcadores de adaptación de cada genoma de la población llamando al método **genoma.setAptitud();**

pruebaRuta() debe comprobar cómo se desplaza por el laberinto desde (fE,0) y devuelve el valor proporcional a la distancia que se queda de la meta. Unas líneas de código que calculen la cercanía podrian ser:

```
int difX = abs(posX – (lab[0].length -1) );
int difY = abs(posY - fM);
```

```
return 1.0/(difX + difY + 1);
```

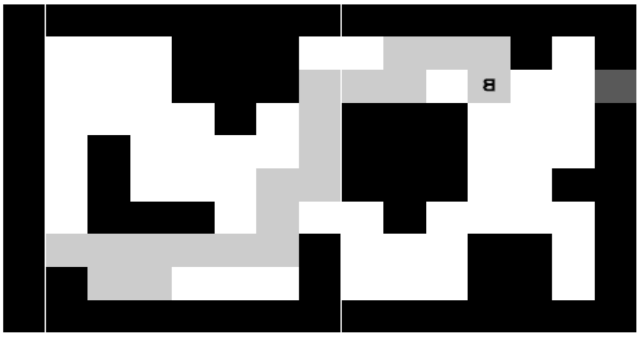


figura 5: Un genoma intenta encontrar la salida.

difX y difY simplemente tienen el número de casillas de distancia hasta la meta en dirección horizontal y vertical. En la figura 5, si las casillas en gris representan la ruta a través del laberinto, y la casilla etiquetada con una B es donde finalmente acaba. En esta posición, difX =3 y difY = 0.

La línea siguiente calcula la aptitud sumando estos dos valores y calculando el inverso. Sumamos 1 al denominador para asegurar de que no dividimos por cero si alcanza la salida.

Además se va a registrar en cada generación (no se pierde de vista) el genoma más apto de cada generación y la aptitud total combinada de todos los genomas (la media de adaptación). Se utilizan estos valores al realizar la selección de la rueda de ruleta.

Como creamos una nueva población de genomas en cada época, necesitamos alguna parte donde ponerlos. Un array de genomas que llamamos **bebes**. El algoritmo de epoca() sería este:

```
generacion <- 0
Poblacion <- crear genomas aleatorios        // Lo puedes hacer tb. en el constructor...
Mientras sinSolucion AND generacion < 1000 Haz
        Calcular adaptación de los genomas de la población.
        Si hay uno con adaptación = 1 Entonces asignarlo a ruta y devolver true; Finsi
        Dibujar el camino del genoma mejor adaptado e indicar valor y nº generación
        NuevosBebes <- 0
        mientras nuevosBebes < poblacion.length haz
                Genoma mami <-  seleccionRuedaRuleta();
                Genoma papi <-  seleccionRuedaRuleta();
                Genoma[] parto <- reproducir(mami, papi);
                Genoma bebe1 <- parto[0]
                Genoma bebe2 <- parto[1]
                aplicaMutacion( bebe1)
                aplicaMutacion( bebe2)
                bebes[nuevosBebes++] <- bebe1
                bebes[nuevosBebes++] <- bebe2
        finMientras
        Sustituir una generación por otra y repetir el proceso
        generacion++
finmientras
```

Vamos a hablar del valor de los parámetros que usamos y que controlan todo el proceso.

Elegir Valores de los Parámetros.

He puesto todos los parámetros usados en el código. La mayoría son auto explicativos, pero

algunos los comento:

```
final double TASA_CRUCE = 0.7;
final double TASA_MUTACION = 0.008;
double TAMA_CROMOSOMA= 2 *(anchoLaberinto + altoLaberinto);
final double TAMA_POBLACION = 2 * TAMA_CROMOSOMA;
```

Podemos preguntarnos cómo sé el valor de las variables que voy a utilizar. Y esa pregunta es la pregunta del millón, porque **no hay unas reglas establecidas** para determinar estos valores, solamente pautas. Que te elijan estos valores supone tener una "sensación" muy pobre de los algoritmos genéticos, y sólo los dominarás cuando juegues con estos parámetros para ver qué sucede. Diferentes problemas necesitan diferentes valores, pero generalmente, si estás utilizando un cromosoma codificado en binario, los **valores 0.7 para tasa de cruce y 0.001 para tasa de mutación son por defecto buenos valores con los que comenzar a probar**.

Una **regla útil para el tamaño inicial de la población es tener el doble de individuos (genomas) de la longitud de un cromosoma**.

Para elegir la longitud de cromosomas en este problema concreto, lo calculas como el doble de la suma de la altura y la longitud del laberinto. En un laberinto de 10x20 puedes comenzar a probar rutas de 2 * (10 + 20) = 60 movimientos -> 2 * 60 = 120 genomas -> comienzas a generar una población de 120 individuos. Y a partir de ahí, hacer pruebas para optimizar el tamaño.

La función de Cruce reproducir().
Se necesita una función que divida a los cromosomas padre en un punto aleatorio y luego intercambie los bits tras este punto para crear dos nuevos cromosomas (los descendientes).

```
String[] reproducir( String mami, String papi) { …
```

Se le pasan dos cromosomas mami (no olvides que un cromosoma es sólo un string de caracteres "NEOS") y crea dos strings vacíos en los que los descendientes se copian.

```
String parto = new String[2];
if ( (Math.random() > TASA_CRUCE) || (mami == papi) ) {    // No hay parto
  parto[0] = mami;
  parto[1] = papi;
  return parto;
}
```

Primero, se comprueba si se va a realizar el cruce de los dos padres, mami y papi. La probabilidad de que ocurra el cruce se basa en TASA_CRUCE. Si no hay cruce, los cromosomas de los padres se copian en los descendientes sin alteración y la función retorna.

De lo contrario, se escoge un punto aleatorio a lo largo de la longitud del cromosoma para dividirlo.

```
ptoCorte = Math.random() * mami.length();
for (int i=0; i < ptoCorte; i++) {
    parto[0]+= mami[i];
    parto[1]+= papi[i];
}
for (i= ptoCorte; i < mami.length(); i++) {
    parto[0]+= papi[i];
    parto[1]+= mami[i];
```

```
}
```

Estos dos pequeños bucles mezclan los genes de cada padre y asigna los nuevos cromosomas a los hijos: parto[0] y parto[1]. Este enfoque siempre genera bebés con genes del mismo tamaño que los padres. Estos genes pueden tener movimientos inútiles (que hagan ...NSNS.., que es como no moverse). **Si quieres que las rutas sean más efectivas (solo tengan movimientos útiles), puedes usar otros enfoques de combinar los genes de padres y madres, pero ten cuidado de no ir perdiendo tamaño en los genes de forma sucesiva o al final no encontrarán una solución.**

La Mutación mutacion()

Esta función simplemente recorre la longitud del cromosoma y cambia los genes con una probabilidad que depende del parámetro TASA_MUTACION.

```
boolean mutacion(Genoma g) {
    boolean mutado= false;
    String ant = g.getGen(), nuevo="";
    for (int i = 0; i < ant.length(); i++) {
        char a = ant.charAt(i);
        if (Math.random() < TASA_MUTACION) {     // ¿Intercambio el bit?
            switch(a) {
             case 'N': a= 'S'; break;
             case 'E': a= 'O'; break;
             case 'S': a= 'N'; break;
             case 'O': a= 'E'; break;
             }
             mutado= true;
        }
        nuevo+= a;
    }
    g.setGen( nuevo );
    return mutado;
}
```

Ejecutando el Programa.

Cuando ejecutas el programa, verás que no siempre encuentra la trayectoria hacia la meta. Lo consigue a veces. Esto es principalmente debido a que a veces la población converge con un tipo particular de cromosoma demasiado rápidamente. Por lo tanto, como la población se convierte en similar, los efectos beneficiosos del operador cruce prácticamente desaparecen, y todos los cambios vienen por una pequeña cantidad de mutaciones. Como la tasa de mutación se pone muy baja, la mutación por sí misma no es suficiente para encontrar una solución una vez que la diversidad de los cromosomas se pierde.

También, se debe a veces a la manera en que actúa la selección, no se garantiza que el cromosoma más apto de cualquier generación pase a la generación siguiente. ¡Esto significa que el algoritmo genético puede encontrar a un miembro de la población que sea casi una solución perfecta, y matarlo más o menos inmediatamente, y al hacer eso, se pierden todos sus buenos genes!

Esta solución propuesta es muy básica y por tanto muy mejorable, es solo una toma de contacto con este tipo de enfoque. Hay técnicas que ayudan a mantener la diversidad de la población y a la vez conservar los genomas más aptos, estrategias para dirigir la evolución hacia un sitio, envejecer a la población en vez de cargártela directamente, ...

2 PUNTOS: Entrega la implementación del algoritmo genético (AG) en una clase, define una variable de esta clase dentro de la clase Laberinto y al constructor del AG le pasas el laberinto (a partir de él define sus parámetros como tamaño de los genomas, de la población y el límite de generaciones que usará en una época). La clase debe tener la función epoca() para que aplique la búsqueda de soluciones. Aplica el primer punto de las sugerencias para jugar con este tipo de programas para que puedas reproducir un mismo escenario al usar valores aleatorios reproducibles varias veces.

1 PUNTO: Entrega 2 capturas de pantalla de ejecutar la misma búsqueda en el mismo laberinto que los algoritmo de backtracking y con el algoritmo genético y compara el comportamiento para laberintos de tamaño (10x20 y prob 0.3) y tamaño (15x60 y prob 0.3).

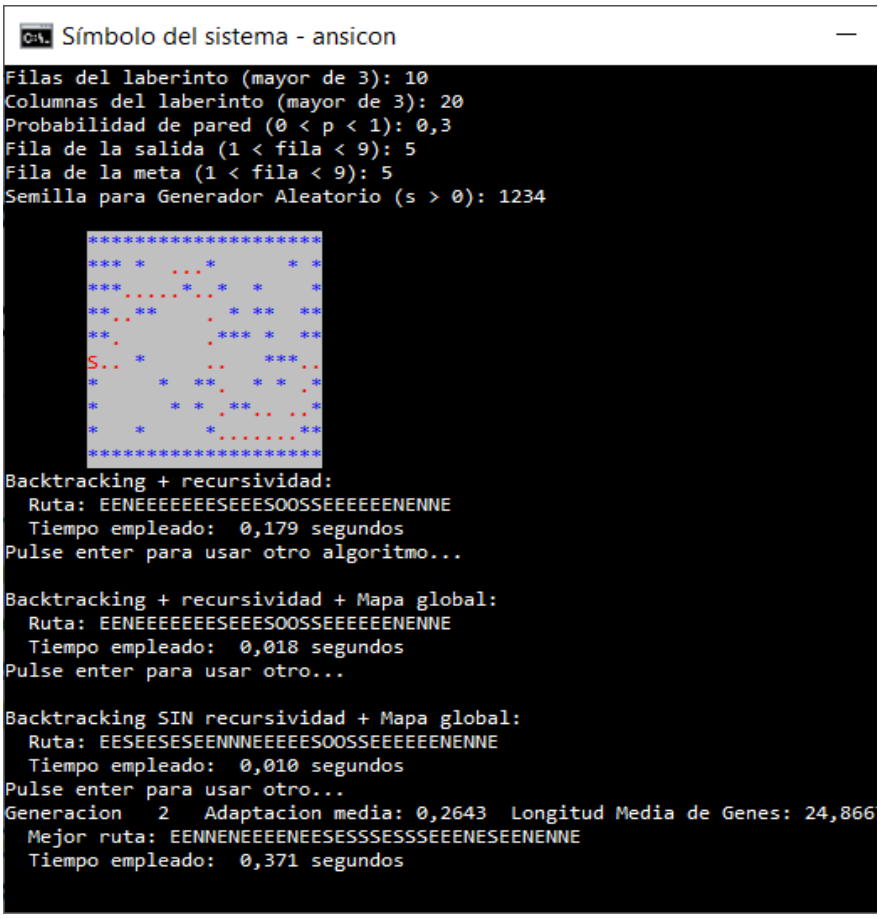


Figura:4 tipos de algoritmos sobre el mismo Laberinto.

En el ejemplo de la figura, el algoritmo usa genes de longitud variable y una vez generados de forma aleatoria los de la población inicial, se purifican dejándolos solo con las instrucciones que realmente se pueden aplicar y que no repiten posiciones. Esto obliga a cambiar la forma de combinar en la reproducción asegurando que no pierdan tamaño generación tras generación.